

Name :- Prof. Vanshika Haribhau Khapekar
Subject :- Operating System

Unit IV

Memory Management, Contiguous allocation, static-swapping, overlays, dynamic partitioned memory allocation, demand paging, page replacement, segmentation. Non-contiguous allocation, paging, Hardware support, Virtual Memory

Memory Management in Operating System

The term memory can be defined as a collection of data in a specific format. It is used to store instructions and process data. The memory comprises a large array or group of words or bytes, each with its own location. The primary purpose of a computer system is to execute programs. These programs, along with the information they access, should be in the main memory during execution. The CPU fetches instructions from memory according to the value of the program counter.

To achieve a degree of multiprogramming and proper utilization of memory, memory management is important. Many memory management methods exist, reflecting various approaches, and the effectiveness of each algorithm depends on the situation.

Here, we will cover the following memory management topics:

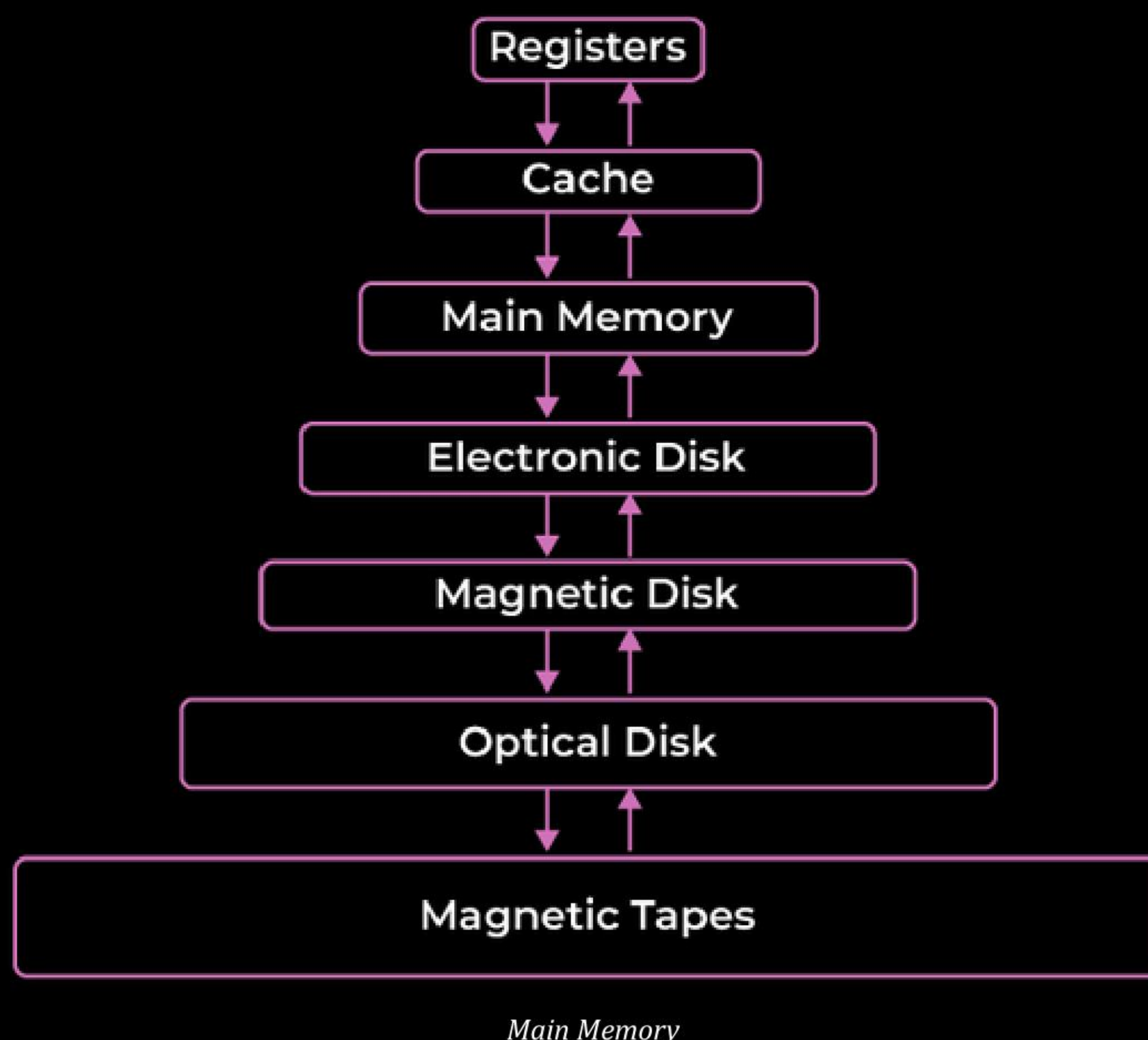
- *What is Main Memory?*
- *What is Memory Management?*
- *Why Memory Management is Required?*
- *Logical Address Space and Physical Address Space*
- *Static and Dynamic Loading*
- *Static and Dynamic Linking*
- *Swapping*
- *Contiguous Memory Allocation*
 - *Memory Allocation*
 - *First Fit*
 - *Best Fit*
 - *Worst Fit*

- *Fragmentation*
 - *Internal Fragmentation*
 - *External Fragmentation*
- *Paging*

Before we start Memory management, let us know what is main memory is.

What is Main Memory?

The main memory is central to the operation of a Modern Computer. Main Memory is a large array of words or bytes, ranging in size from hundreds of thousands to billions. Main memory is a repository of rapidly available information shared by the CPU and I/O devices. Main memory is the place where programs and information are kept when the processor is effectively utilizing them. Main memory is associated with the processor, so moving instructions and information into and out of the processor is extremely fast. Main memory is also known as RAM (Random Access Memory). This memory is volatile. RAM loses its data when a power interruption occurs.



What is Memory Management?

In a multiprogramming computer, the Operating System resides in a part of memory, and the rest is used by multiple processes. The task of subdividing the memory among different processes is called Memory Management. Memory management is a method in the operating system to manage

operations between main memory and disk during process execution. The main aim of memory management is to achieve efficient utilization of memory.

Why Memory Management is Required?

- Allocate and de-allocate memory before and after process execution.
- To keep track of used memory space by processes.
- To minimize fragmentation issues.
- To proper utilization of main memory.
- To maintain data integrity while executing of process.

Now we are ing the concept of Logical Address Space and Physical Address Space

Logical and Physical Address Space

- **Logical Address Space:** An address generated by the CPU is known as a “Logical Address”. It is also known as a Virtual address. Logical address space can be defined as the size of the process. A logical address can be changed.
- **Physical Address Space:** An address seen by the memory unit (i.e the one loaded into the memory address register of the memory) is commonly known as a “Physical Address”. A Physical address is also known as a Real address. The set of all physical addresses corresponding to these logical addresses is known as Physical address space. A physical address is computed by MMU. The run-time mapping from virtual to physical addresses is done by a hardware device Memory Management Unit(MMU). The physical address always remains constant.

Static and Dynamic Loading

Loading a process into the main memory is done by a loader. There are two different types of loading :

- **Static Loading:** Static Loading is basically loading the entire program into a fixed address. It requires more memory space.
- **Dynamic Loading:** The entire program and all data of a process must be in physical memory for the process to execute. So, the size of a process is limited to the size of physical memory. To gain proper memory utilization, dynamic loading is used. In dynamic loading, a routine is not loaded until it is called. All routines are residing on disk in a relocatable load format. One of the advantages of dynamic loading is that the unused routine is never loaded. This loading is useful when a large amount of code is needed to handle it efficiently.

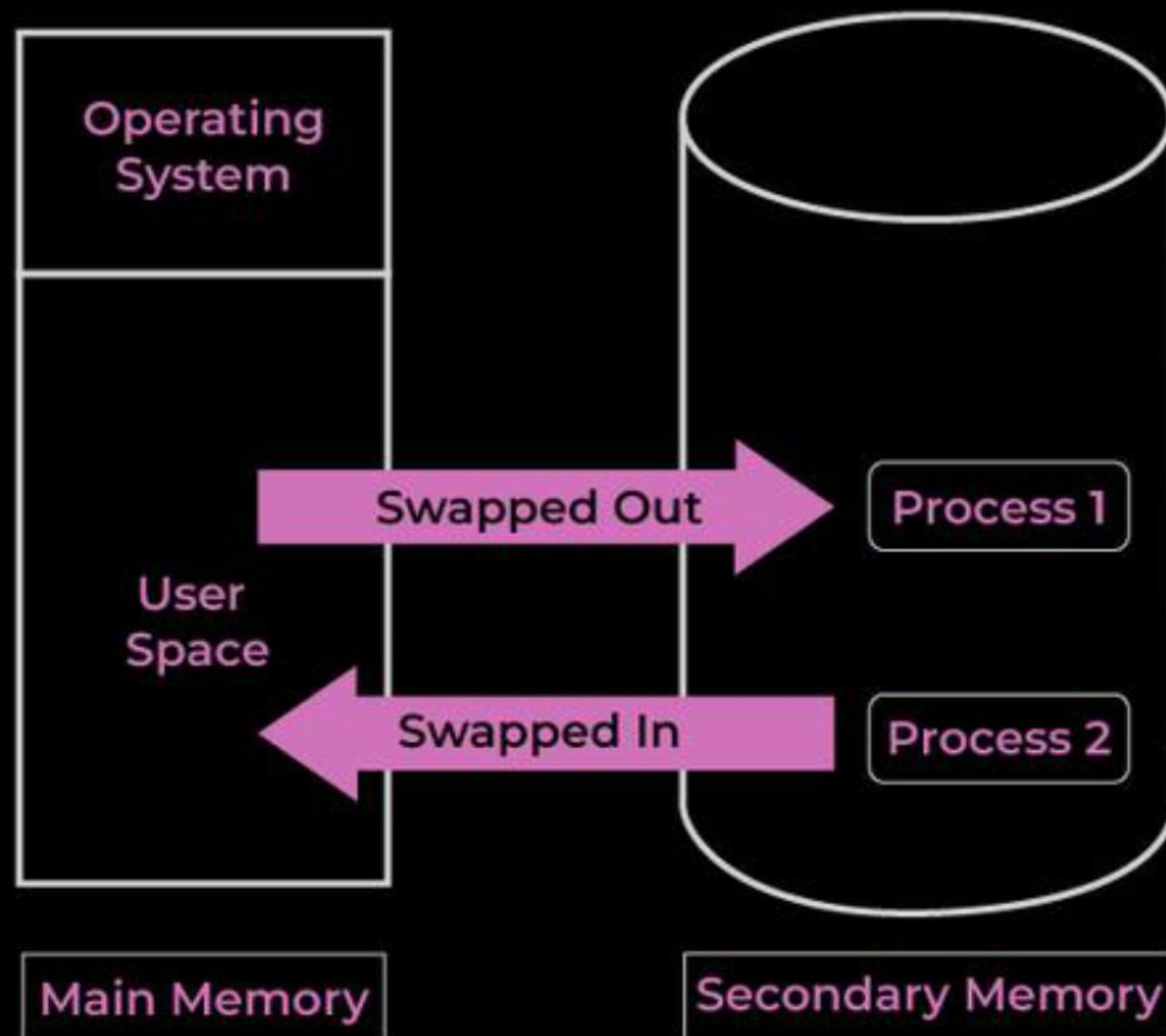
Static and Dynamic Linking

To perform a linking task a linker is used. A linker is a program that takes one or more object files generated by a compiler and combines them into a single executable file.

- **Static Linking:** In static linking, the linker combines all necessary program modules into a single executable program. So there is no runtime dependency. Some operating systems support only static linking, in which system language libraries are treated like any other object module.
- **Dynamic Linking:** The basic concept of dynamic linking is similar to dynamic loading. In dynamic linking, “Stub” is included for each appropriate library routine reference. A stub is a small piece of code. When the stub is executed, it checks whether the needed routine is already in memory or not. If not available then the program loads the routine into memory.

Swapping

When a process is executed it must have resided in memory. Swapping is a process of swapping a process temporarily into a secondary memory from the main memory, which is fast compared to secondary memory. A swapping allows more processes to be run and can be fit into memory at one time. The main part of swapping is transferred time and the total time is directly proportional to the amount of memory swapped. Swapping is also known as roll-out, or roll because if a higher priority process arrives and wants service, the memory manager can swap out the lower priority process and then load and execute the higher priority process. After finishing higher priority work, the lower priority process swapped back in memory and continued to the execution process.



swapping in memory management

Memory Management with Monoprogramming (Without Swapping)

This is the simplest memory management approach the memory is divided into two sections:

- One part of the operating system
- The second part of the user program

Fence Register	
operating system	user program

- In this approach, the operating system keeps track of the first and last location available for the allocation of the user program
- The operating system is loaded either at the bottom or at top
- Interrupt vectors are often loaded in low memory therefore, it makes sense to load the operating system in low memory
- Sharing of data and code does not make much sense in a single process environment
- The Operating system can be protected from user programs with the help of a fence register.

Advantages of Memory Management

- It is a simple management approach

Disadvantages of Memory Management

- It does not support multiprogramming

- Memory is wasted

Multiprogramming with Fixed Partitions (Without Swapping)

- A memory partition scheme with a fixed number of partitions was introduced to support multiprogramming. this scheme is based on contiguous allocation
- Each partition is a block of contiguous memory
- Memory is partitioned into a fixed number of partitions.
- Each partition is of fixed size

Example: As shown in fig. memory is partitioned into 5 regions the region is reserved for updating the system the remaining four partitions are for the user program.

Fixed Size Partitioning

Operating System
p1
p2
p3
p4

Partition Table

Once partitions are defined operating system keeps track of the status of memory partitions it is done through a data structure called a partition table.

Sample Partition Table

Starting Address of Partition	Size of Partition	Status
0k	200k	allocated
200k	100k	free
300k	150k	free
450k	250k	allocated

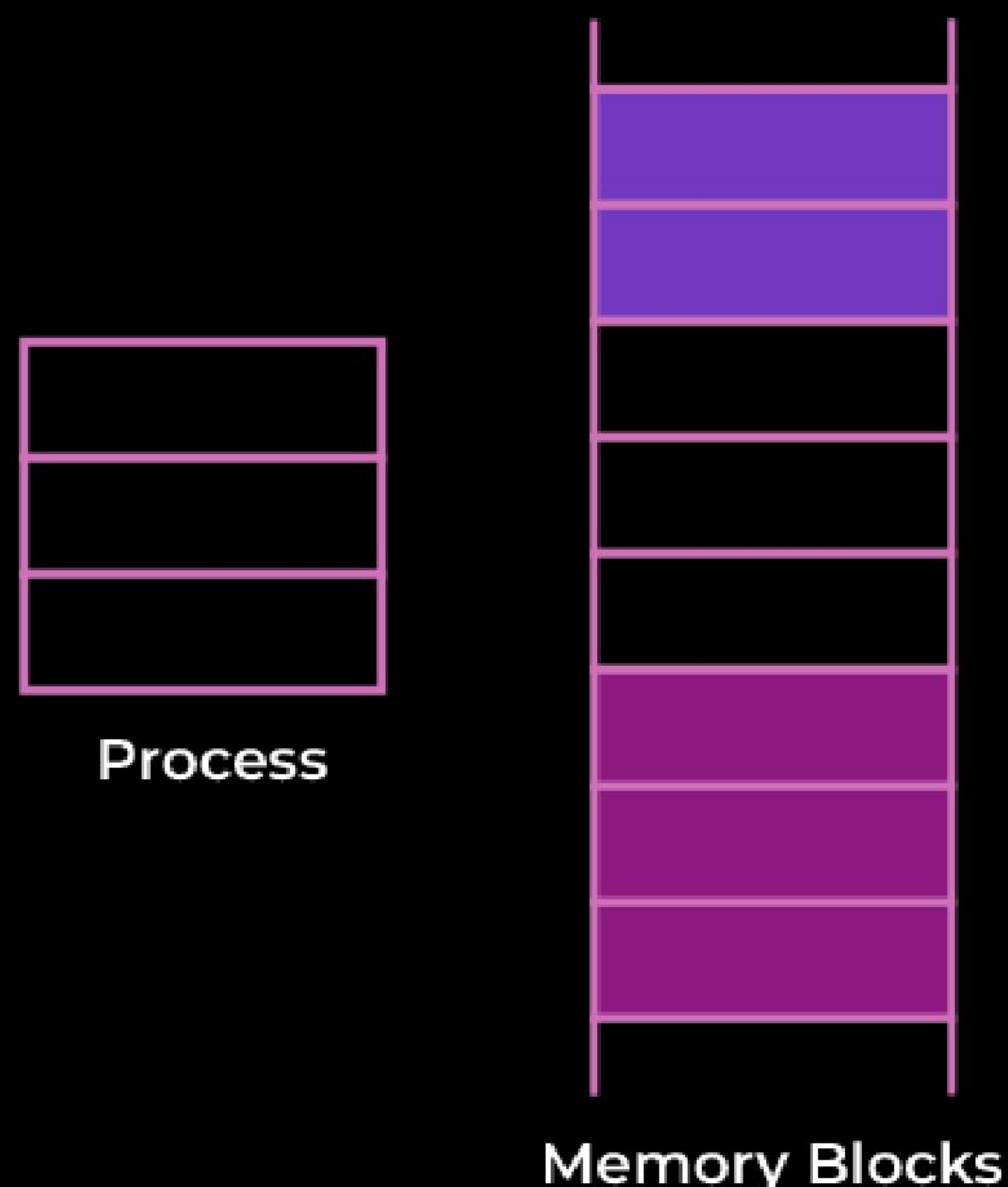
Logical vs Physical Address

An address generated by the CPU is commonly referred to as a logical address. the address seen by the memory unit is known as the physical address. The logical address can be mapped to a physical address by

hardware with the help of a base register this is known as dynamic relocation of memory references.

Contiguous Memory Allocation

The main memory should accommodate both the operating system and the different client processes. Therefore, the allocation of memory becomes an important task in the operating system. The memory is usually divided into two partitions: one for the resident operating system and one for the user processes. We normally need several user processes to reside in memory simultaneously. Therefore, we need to consider how to allocate available memory to the processes that are in the input queue waiting to be brought into memory. In adjacent memory allotment, each process is contained in a single contiguous segment of memory.



Contiguous Memory Allocation

Memory Allocation

To gain proper memory utilization, memory allocation must be allocated efficient manner. One of the simplest methods for allocating memory is to divide memory into several fixed-sized partitions and each partition

contains exactly one process. Thus, the degree of multiprogramming is obtained by the number of partitions.

- **Multiple partition allocation:** In this method, a process is selected from the input queue and loaded into the free partition. When the process terminates, the partition becomes available for other processes.
- **Fixed partition allocation:** In this method, the operating system maintains a table that indicates which parts of memory are available and which are occupied by processes. Initially, all memory is available for user processes and is considered one large block of available memory. This available memory is known as a "Hole". When the process arrives and needs memory, we search for a hole that is large enough to store this process. If the requirement is fulfilled then we allocate memory to process, otherwise keeping the rest available to satisfy future requests. While allocating a memory sometimes dynamic storage allocation problems occur, which concerns how to satisfy a request of size n from a list of free holes. There are some solutions to this problem:

First Fit

In the First Fit, the first available free hole fulfil the requirement of the process allocated.



First Fit

Here, in this diagram, a 40 KB memory block is the first available free hole that can store process A (size of 25 KB), because the first two blocks did not have sufficient memory space.

Best Fit

In the Best Fit, allocate the smallest hole that is big enough to process requirements. For this, we search the entire list, unless the list is ordered by size.

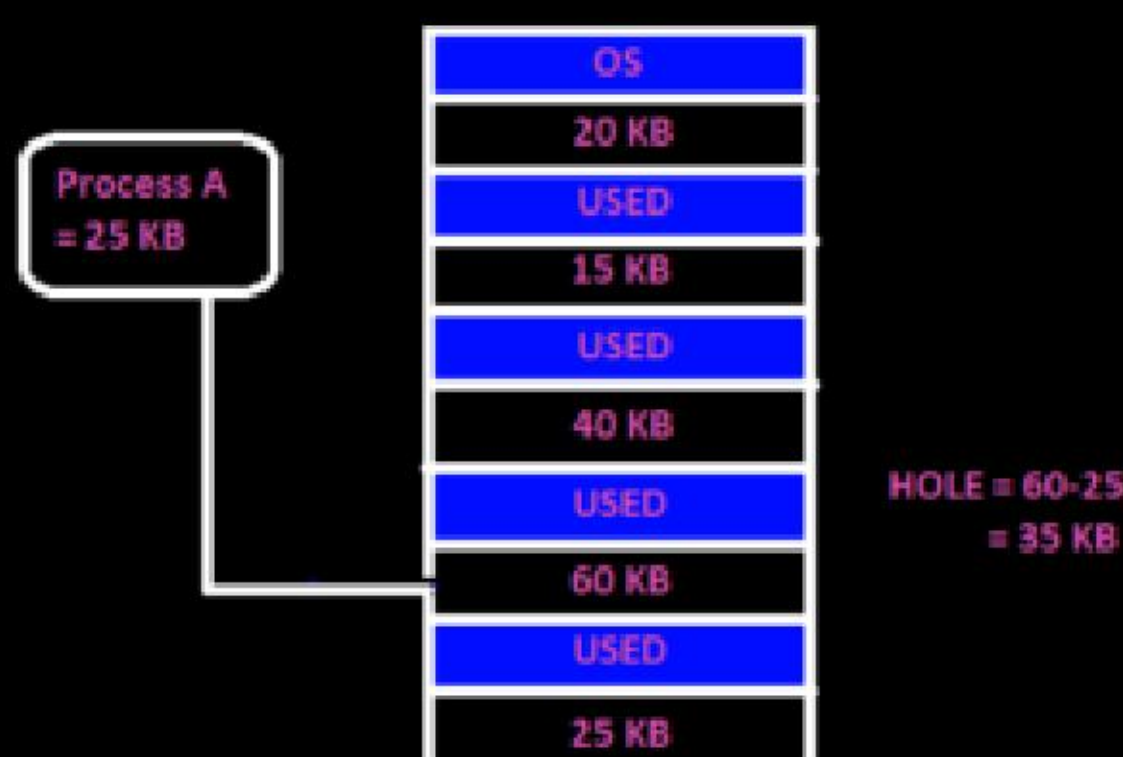


Best Fit

Here in this example, first, we traverse the complete list and find the last hole 25KB is the best suitable hole for Process A(size 25KB). In this method, memory utilization is maximum as compared to other memory allocation techniques.

Worst Fit

In the Worst Fit, allocate the largest available hole to process. This method produces the largest leftover hole.



Worst Fit

Here in this example, Process A (Size 25 KB) is allocated to the largest available memory block which is 60KB. Inefficient memory utilization is a major issue in the worst fit.

Fragmentation

Fragmentation is defined as when the process is loaded and removed after execution from memory, it creates a small free hole. These holes can not be assigned to new processes because holes are not combined or do not fulfill the memory requirement of the process. To achieve a degree of multiprogramming, we must reduce the waste of memory or fragmentation problems. In the operating systems two types of fragmentation:

1. **Internal fragmentation:** Internal fragmentation occurs when memory blocks are allocated to the process more than their requested size. Due to this some unused space is left over and creating an internal fragmentation problem. **Example:** Suppose there is a fixed partitioning used for memory allocation and the different sizes of blocks 3MB, 6MB, and 7MB space in memory. Now a new process p4 of size 2MB comes and demands a block of memory. It gets a memory block of 3MB but 1MB block of memory is a waste, and it can not be allocated to other processes too. This is called internal fragmentation.
2. **External fragmentation:** In External Fragmentation, we have a free memory block, but we can not assign it to a process because blocks are not contiguous. **Example:** Suppose (consider the above example) three processes p1, p2, and p3 come with sizes 2MB, 4MB, and 7MB respectively. Now they get memory blocks of size 3MB, 6MB, and 7MB allocated respectively. After allocating the process p1 process and the p2 process left 1MB and 2MB. Suppose a new process p4 comes and demands a 3MB block of memory, which is available, but we can not assign it because free memory space is not contiguous. This is called external fragmentation.

Both the first-fit and best-fit systems for memory allocation are affected by external fragmentation. To overcome the external fragmentation problem Compaction is used. In the compaction technique, all free memory space combines and makes one large block. So, this space can be used by other processes effectively.

Another possible solution to the external fragmentation is to allow the logical address space of the processes to be noncontiguous, thus permitting a process to be allocated physical memory wherever the latter is available.

Paging

Paging is a memory management scheme that eliminates the need for a contiguous allocation of physical memory. This scheme permits the physical address space of a process to be non-contiguous.

- **Logical Address or Virtual Address (represented in bits):** An address generated by the CPU.
- **Logical Address Space or Virtual Address Space (represented in words or bytes):** The set of all logical addresses generated by a program.
- **Physical Address (represented in bits):** An address actually available on a memory unit.
- **Physical Address Space (represented in words or bytes):** The set of all physical addresses corresponding to the logical addresses.

Example:

- If Logical Address = 31 bits, then Logical Address Space = 2^{31} words = 2 G words (1 G = 2^{30})
- If Logical Address Space = 128 M words = $2^7 * 2^{20}$ words, then Logical Address = $\log_2 2^{27} = 27$ bits
- If Physical Address = 22 bits, then Physical Address Space = 2^{22} words = 4 M words (1 M = 2^{20})
- If Physical Address Space = 16 M words = $2^4 * 2^{20}$ words, then Physical Address = $\log_2 2^{24} = 24$ bits

The mapping from virtual to physical address is done by the memory management unit (MMU) which is a hardware device and this mapping is known as the paging technique.

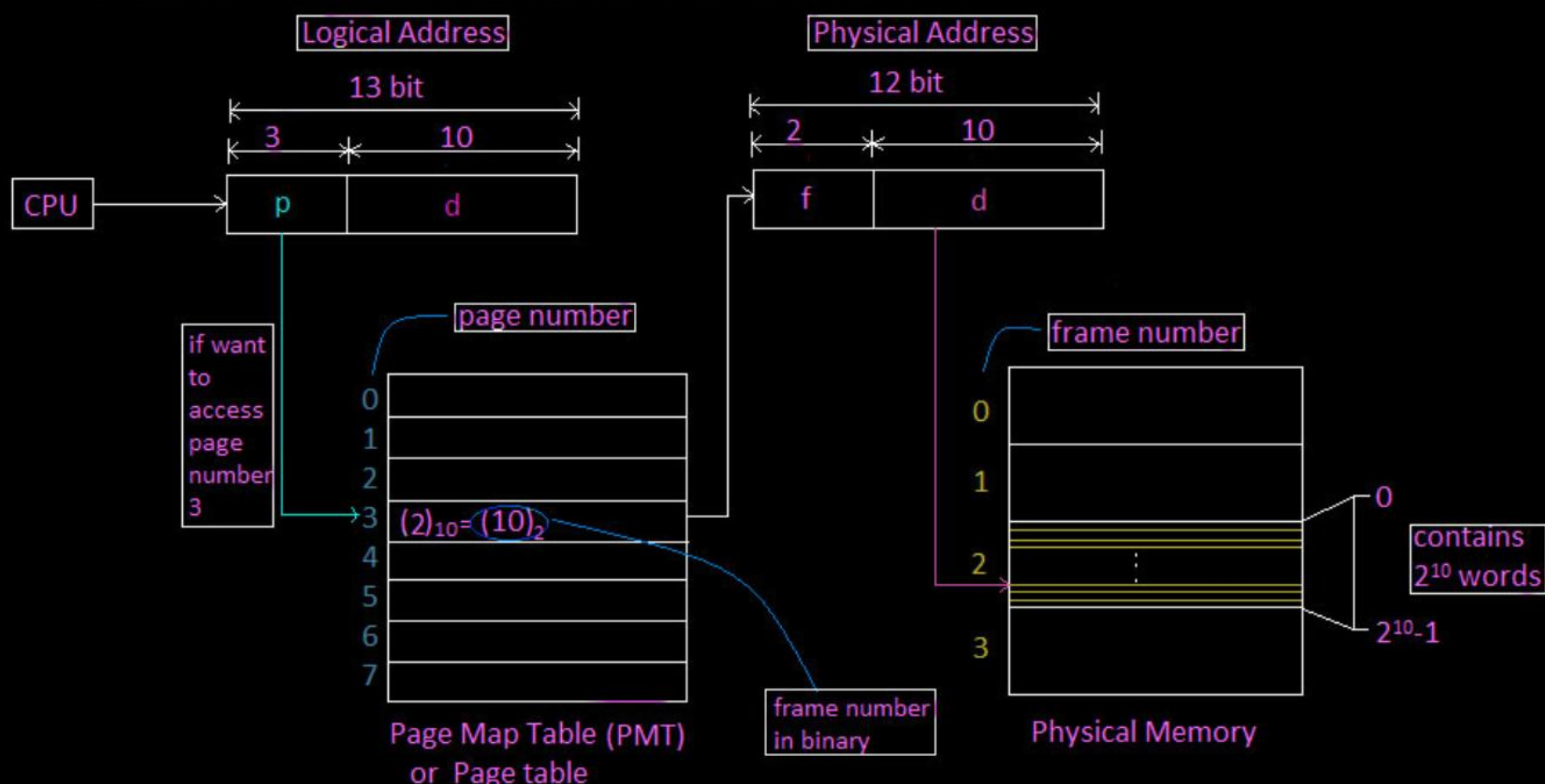
- The Physical Address Space is conceptually divided into several fixed-size blocks, called **frames**.
- The Logical Address Space is also split into fixed-size blocks, called **pages**.
- Page Size = Frame Size

Let us consider an example:

- Physical Address = 12 bits, then Physical Address Space = 4 K words
- Logical Address = 13 bits, then Logical Address Space = 8 K words
- Page size = frame size = 1 K words (assumption)

$$\text{Number of frames} = \text{Physical Address Space} / \text{Frame size} = 4 \text{ K} / 1 \text{ K} = 4 = 2^2$$

$$\text{Number of pages} = \text{Logical Address Space} / \text{Page size} = 8 \text{ K} / 1 \text{ K} = 8 = 2^3$$



Paging

The address generated by the CPU is divided into:

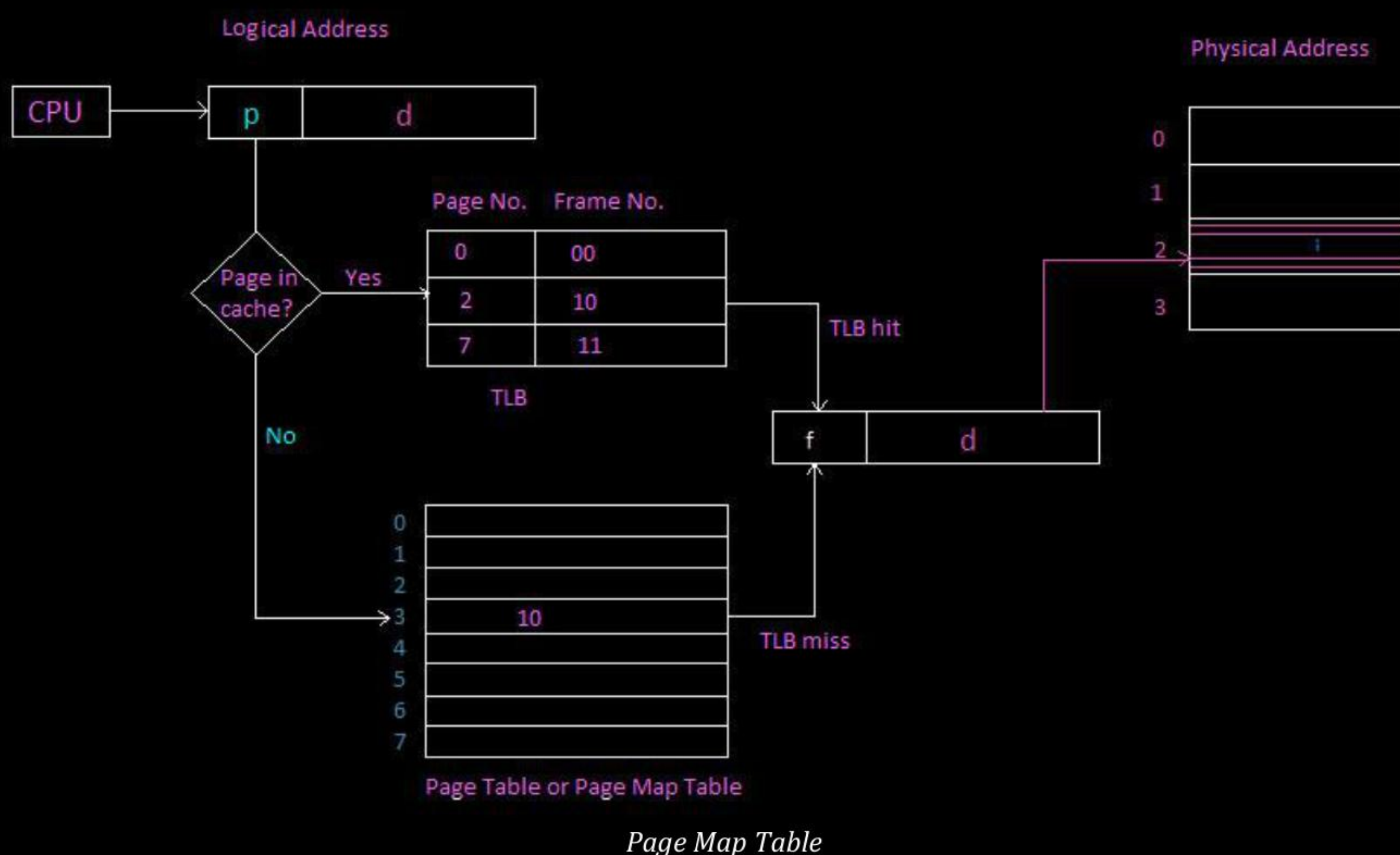
- **Page Number(p):** Number of bits required to represent the pages in Logical Address Space or Page number
- **Page Offset(d):** Number of bits required to represent a particular word in a page or page size of Logical Address Space or word number of a page or page offset.

Physical Address is divided into:

- **Frame Number(f):** Number of bits required to represent the frame of Physical Address Space or Frame number frame
- **Frame Offset(d):** Number of bits required to represent a particular word in a frame or frame size of Physical Address Space or word number of a frame or frame offset.

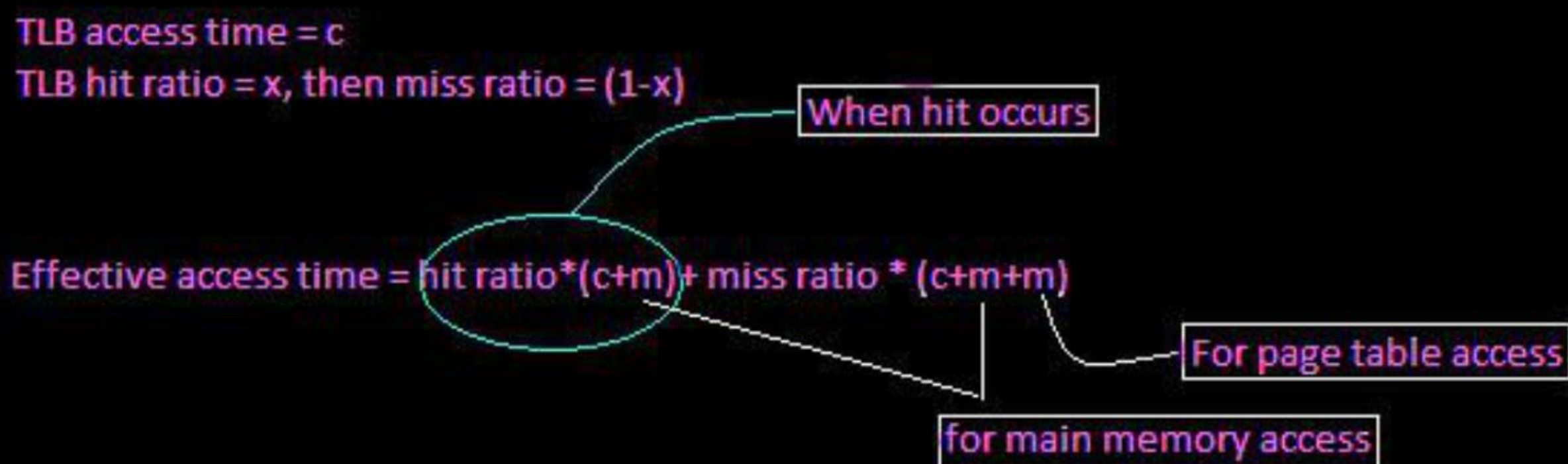
The hardware implementation of the page table can be done by using dedicated registers. But the usage of the register for the page table is satisfactory only if the page table is small. If the page table contains a large number of entries then we can use TLB(translation Look-aside buffer), a special, small, fast look-up hardware cache.

- The TLB is an associative, high-speed memory.
- Each entry in TLB consists of two parts: a tag and a value.
- When this memory is used, then an item is compared with all tags simultaneously. If the item is found, then the corresponding value is returned.



Main memory access time = m

If page table are kept in main memory,
Effective access time = m(for page table)
+ m(for particular page in page table)



TLB Hit and Miss

Frequently Asked Questions

Q.1: What is a memory leak, and how does it affect system performance?

Answers:

A memory leak occurs when a program fails to release memory that it no longer needs, resulting in wasted memory resources. Over time, if memory leaks accumulate, the system's available memory diminishes, leading to reduced performance and possibly system crashes.

Q.2: Can memory fragmentation be prevented in an operating system?

Answers:

While it is challenging to completely eliminate memory fragmentation, certain techniques can help minimize its impact. One approach is to use memory allocation algorithms that focus on reducing external fragmentation, such as buddy systems or slab allocation. Additionally, techniques like compaction can be employed to reduce internal fragmentation.

Q.3: What are the advantages and disadvantages of using virtual memory?

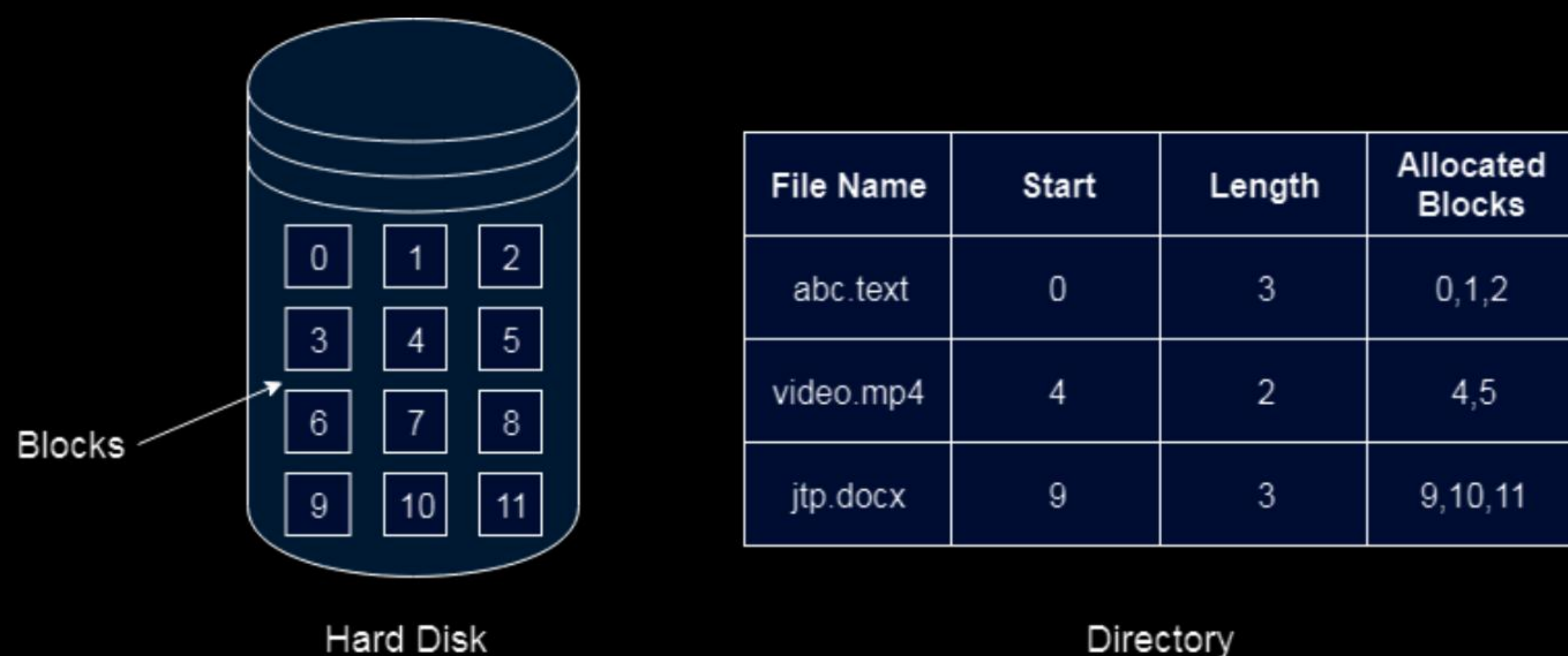
Answers:

Virtual memory provides several benefits, including the ability to run larger programs than the available physical memory, increased security through memory isolation, and simplified memory management for applications. However, using virtual memory also introduces additional overhead due to page table lookups and potential performance degradation if excessive swapping occurs.

Contiguous Allocation

If the blocks are allocated to the file in such a way that all the logical blocks of the file get the contiguous physical block in the hard disk then such allocation scheme is known as contiguous allocation.

In the image shown below, there are three files in the directory. The starting block and the length of each file are mentioned in the table. We can check in the table that the contiguous blocks are assigned to each file as per its need.



Contiguous Allocation

Advantages

1. It is simple to implement.
2. We will get Excellent read performance.
3. Supports Random Access into files.

Disadvantages

1. The disk will become fragmented.
2. It may be difficult to have a file grow.

Swapping in Operating System

Swapping is a memory management scheme in which any process can be temporarily swapped from main memory to secondary memory so that the main memory can be made available for other processes. It is used to improve main memory utilization. In secondary memory, the place where the swapped-out process is stored is called swap space.

The purpose of the swapping in operating system is to access the data present in the hard disk and bring it to RAM so that the application programs can use it. The thing to remember is that swapping is used only when data is not present in RAM.

Although the process of swapping affects the performance of the system, it helps to run larger and more than one process. This is the reason why swapping is also referred to as memory compaction.

The concept of swapping has divided into two more concepts: Swap-in and Swap-out.

- Swap-out is a method of removing a process from RAM and adding it to the hard disk.
- Swap-in is a method of removing a program from a hard disk and putting it back into the main memory or RAM.

Example: Suppose the user process's size is 2048KB and is a standard hard disk where swapping has a data transfer rate of 1Mbps. Now we will calculate how long it will take to transfer from main memory to secondary memory.

1. User process size is 2048Kb
2. Data transfer rate is 1Mbps = 1024 kbps
3. Time = process size / transfer rate
4. = 2048 / 1024
5. = 2 seconds
6. = 2000 milliseconds
7. Now taking swap-in and swap-out time, the process will take 4000 milliseconds.

Advantages of Swapping

1. It helps the CPU to manage multiple processes within a single main memory.
2. It helps to create and use virtual memory.
3. Swapping allows the CPU to perform multiple tasks simultaneously. Therefore, processes do not have to wait very long before they are executed.

4. It improves the main memory utilization.

Disadvantages of Swapping

1. If the computer system loses power, the user may lose all information related to the program in case of substantial swapping activity.
2. If the swapping algorithm is not good, the composite method can increase the number of Page Fault and decrease the overall processing performance.

Note:

- In a single tasking operating system, only one process occupies the user program area of memory and stays in memory until the process is complete.
- In a multitasking operating system, a situation arises when all the active processes cannot coordinate in the main memory, then a process is swap out from the main memory so that other processes can enter it.

Overlays in Memory Management

In memory management, overlays refer to a technique used to manage memory efficiently by overlaying a portion of memory with another program or data.

The idea behind overlays is to only load the necessary parts of a program into memory at a given time, freeing up memory for other tasks. The unused portions of the program are kept on disk or other storage, and are loaded into memory as needed. This allows programs to be larger than the available memory, but still run smoothly.

The main problem in Fixed partitioning is the size of a process has to be limited by the maximum size of the partition, which means a process can never be span over another. In order to solve this problem, earlier people have used some solution which is called as Overlays.

The concept of **overlays** is that whenever a process is running it will not use the complete program at the same time, it will use only some part of it. Then overlays concept says that whatever part you required, you load it and once the part is done, then you just unload it, means just pull it back and get the new part you required and run it.

Formally,

“The process of **transferring a block** of program code or other data into internal memory, replacing what is already stored”.

Sometimes it happens that compare to the size of the biggest partition, the size of the program will be even more, then, in that case, you should go with overlays.

So overlay is a technique to run a program that is bigger than the size of the physical memory by keeping only those instructions and data that are needed at any given time. Divide the program into modules in such a way that not all modules need to be in the memory at the same time.

Advantages of using overlays include:

- Increased memory utilization: Overlays allow multiple programs to share the same physical memory space, increasing memory utilization and reducing the need for additional memory.
- Reduced load time: Only the necessary parts of a program are loaded into memory, reducing load time and increasing performance.
- Improved reliability: Overlays reduce the risk of memory overflow, which can cause crashes or data loss.
- Reduce memory requirement
- Reduce time requirement

Disadvantages of using overlays include:

- Complexity: Overlays can be complex to implement and manage, especially for large programs.
- Performance overhead: The process of loading and unloading overlays can result in increased CPU and disk usage, which can slow down performance.
- Compatibility issues: Overlays may not work on all hardware and software configurations, making it difficult to ensure compatibility across different systems.
- Overlap map must be specified by programmer
- Programmer must know memory requirement
- Overlapped module must be completely disjoint
- Programming design of overlays structure is complex and not possible in all cases

Example –

The best example of overlays is assembler. Consider the assembler has 2 passes, 2 pass means at any time it will be doing only one thing, either the 1st pass or the 2nd pass. This means it will finish 1st pass first and then 2nd pass. Let assume that available main memory size is 150KB and total code size is 200KB

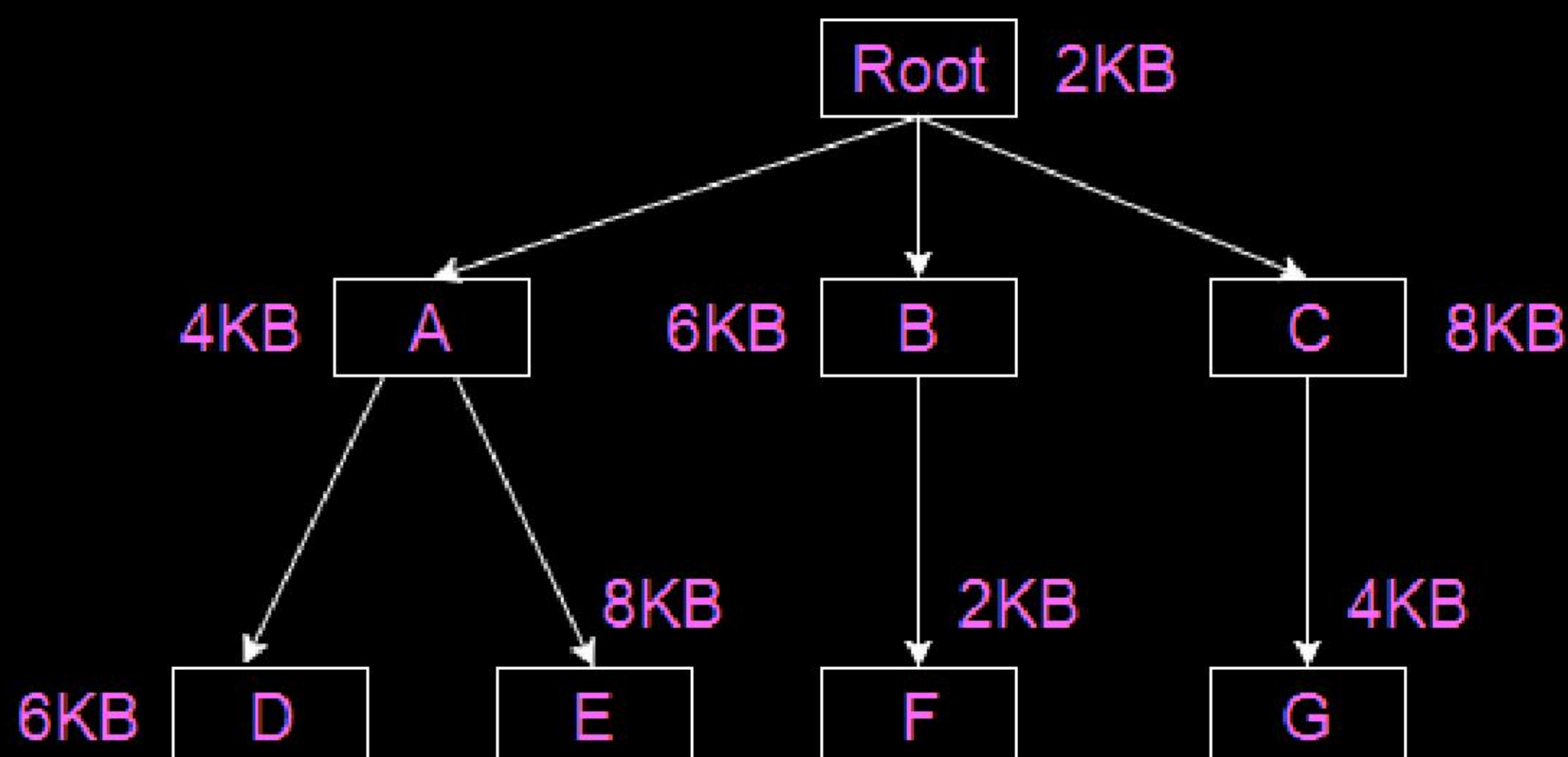
Pass 1.....70KB
 Pass 2.....80KB
 Symbol table.....30KB
 Common routine.....20KB

As the total code size is 200KB and main memory size is 150KB, it is not possible to use 2 passes together. So, in this case, we should go with the overlays technique. According to the overlays concept at any time only one pass will be used and both the passes always need symbol table and common routine. Now the question is if overlays-driver* is 10KB, then what is the minimum partition size required? For pass 1 total memory needed is = (70KB + 30KB + 20KB + 10KB) = 130KB and for pass 2 total memory needed is = (80KB + 30KB + 20KB + 10KB) = 140KB. So if we have minimum 140KB size partition then we can run this code very easily.

*Overlays driver:-It is the user responsibility to take care of overlaying, the operating system will not provide anything. Which means the user should write even what part is required in the 1st pass and once the 1st pass is over, the user should write the code to pull out the pass 1 and load the pass 2. That is what is the responsibility of the user, that is known as the Overlays driver. Overlays driver will just help us to move out and move in the various part of the code.

Question –

The overlay tree for a program is as shown below:



What will be the size of the partition (in physical memory) required to load (and run) this program?

(a) 12 KB (b) 14 KB (c) 10 KB (d) 8 KB

Explanation –

Using the overlay concept we need not actually have the entire program inside the main memory. Only we need to have the part which are required at that instance of time, either we need Root-A-D or Root-A-E or Root-B-F or Root-C-G part.

$$\text{Root+A+D} = 2\text{KB} + 4\text{KB} + 6\text{KB} = 12\text{KB}$$

$$\text{Root+A+E} = 2\text{KB} + 4\text{KB} + 8\text{KB} = 14\text{KB}$$

$$\text{Root+B+F} = 2\text{KB} + 6\text{KB} + 2\text{KB} = 10\text{KB}$$

$$\text{Root+C+G} = 2\text{KB} + 8\text{KB} + 4\text{KB} = 14\text{KB}$$

So if we have 14KB size of partition then we can run any of them.

Answer -(b) 14KB

Variable (or dynamic) Partitioning in Operating System

In operating systems, Memory Management is the function responsible for allocating and managing a computer's main memory. The memory Management function keeps track of the status of each memory location, either allocated or free to ensure effective and efficient use of Primary Memory.

Types of Memory Management Techniques

- Contiguous
- Non-Contiguous

In Contiguous Technique, executing process must be loaded entirely in the main memory. The contiguous Technique can be divided into:

1. Fixed (or static) partitioning
2. Variable (or dynamic) partitioning

Variable (or Dynamic) Partitioning

It is a part of the Contiguous allocation technique. It is used to alleviate the problem faced by Fixed Partitioning. In contrast with fixed partitioning, partitions are not made before the execution or during system configuration.

Various **features** associated with variable Partitioning-

- Initially, RAM is empty and partitions are made during the run-time according to the process's need instead of partitioning during system configuration.
- The size of the partition will be equal to the incoming process.

- The partition size varies according to the need of the process so that internal fragmentation can be avoided to ensure efficient utilization of RAM.
- The number of partitions in RAM is not fixed and depends on the number of incoming processes and the Main Memory's size.

Dynamic partitioning

Operating system	
P1 = 2 MB	Block size = 2 MB
P2 = 7 MB	Block size = 7 MB
P3 = 1 MB	Block size = 1 MB
P4 = 5 MB	Block size = 5 MB
Empty space of RAM	

Partition size = process size
So, no internal Fragmentation

Dynamic Partitioning

Advantages of Variable Partitioning

- **No Internal Fragmentation:** In variable Partitioning, space in the main memory is allocated strictly according to the need of the process, hence there is no case of internal fragmentation. There will be no unused space left in the partition.
- **No restriction on the Degree of Multiprogramming:** More processes can be accommodated due to the absence of internal fragmentation. A process can be loaded until the memory is empty.
- **No Limitation on the Size of the Process:** In Fixed partitioning, the process with a size greater than the size of the largest partition could not be loaded and the process can not be divided as it is invalid in the contiguous allocation technique. Here, In variable partitioning, the process size can't be restricted since the partition size is decided according to the process size.

Disadvantages of Variable Partitioning

- **Difficult Implementation:** Implementing variable Partitioning is difficult as compared to Fixed Partitioning as it involves the allocation of memory during run-time rather than during system configuration.
- **External Fragmentation:** There will be external fragmentation despite the absence of internal fragmentation. For example, suppose in the above example- process P1(2MB) and process P3(1MB) completed their execution. Hence two spaces are left i.e. 2MB and 1MB. Let's suppose process P5 of size 3MB comes. The space in memory cannot be allocated as no spanning is allowed in contiguous allocation. The rule says that the process must be continuously present in the main memory to get executed. Hence it results in External Fragmentation.

Dynamic partitioning

Operating system	
P1 (2 MB) executed, now empty	Block size = 2 MB
P2 = 7 MB	Block size = 7 MB
P3 (1 MB) executed	Block size = 1 MB
P4 = 5 MB	Block size = 5 MB
Empty space of RAM	

Partition size = process size
So, no internal Fragmentation

No Internal Fragmentation

Now P5 of size 3 MB cannot be accommodated despite the required available space because in contiguous no spanning is allowed.

Key Points On Variable (or Dynamic) Partitioning in Operating Systems

- Variable (or dynamic) partitioning is a memory allocation technique that allows memory partitions to be created and resized dynamically as needed.
- The operating system maintains a table of free memory blocks or holes, each of which represents a potential partition. When a

process requests memory, the operating system searches the table for a suitable hole that can accommodate the requested amount of memory.

- Dynamic partitioning reduces internal fragmentation by allocating memory more efficiently, allows multiple processes to share the same memory space, and is flexible in accommodating processes with varying memory requirements.
- However, dynamic partitioning can also lead to external fragmentation and requires more complex memory management algorithms, which can make it slower than fixed partitioning.
- Understanding dynamic partitioning is essential for operating system design and implementation, as well as for system-level programming.

What is Demand Paging in Operating System?

The concept of query navigation in the operating system. This concept says that we should not load any pages into the main memory until we need them, or keep all pages in secondary memory until we need them.

Demand Paging

Demand paging can be described as a memory management technique that is used in operating systems to improve memory usage and system performance. Demand paging is a technique used in virtual memory systems where pages enter main memory only when requested or needed by the CPU. In demand paging, the operating system loads only the necessary pages of a program into memory at runtime, instead of loading the entire program into memory at the start.

A page fault occurred when the program needed to access a page that is not currently in memory. The operating system then loads the required pages from the disk into memory and updates the page tables accordingly. This process is transparent to the running program and it continues to run as if the page had always been in memory.

Pure Demand Paging

Pure demand paging is a specific implementation of demand paging. The operating system only loads pages into memory when the program needs them. In on-demand paging only, no pages are initially loaded into memory when the program starts, and all pages are initially marked as being on disk.

Benefits of the Demand Paging

So in the Demand Paging technique, there are some benefits that provide efficiency of the operating system.

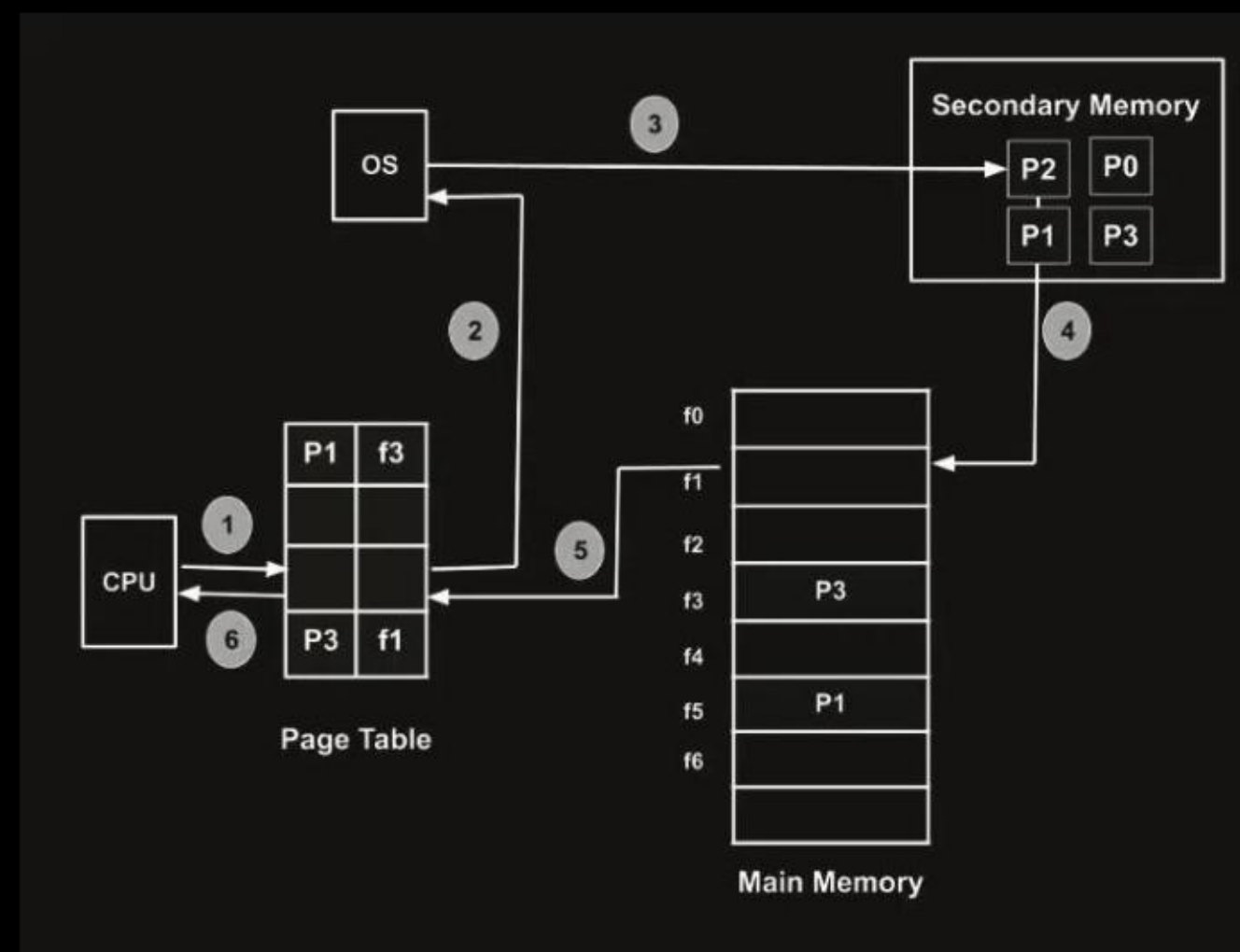
- **Efficient use of physical memory:** Query paging allows for more efficient use because only the necessary pages are loaded into memory at any given time.
- **Support for larger programs:** Programs can be larger than the physical memory available on the system because only the necessary pages will be loaded into memory.
- **Faster program start:** Because only part of a program is initially loaded into memory, programs can start faster than if the entire program were loaded at once.
- **Reduce memory usage:** Query paging can help reduce the amount of memory a program needs, which can improve system performance by reducing the amount of disk I/O required.

Drawbacks of the Demand Paging

- **Page Fault Overload:** The process of swapping pages between memory and disk can cause a performance overhead, especially if the program frequently accesses pages that are not currently in memory.
- **Degraded performance:** If a program frequently accesses pages that are not currently in memory, the system spends a lot of time swapping out pages, which degrades performance.
- **Fragmentation:** Query paging can cause physical memory fragmentation, degrading system performance over time.
- **Complexity:** Implementing query paging in an operating system can be complex, requiring complex algorithms and data structures to manage page tables and swap space.

Working Process of Demand Paging

So, let's understand this with the help of an example. Suppose we want to run a process P which has four pages P0, P1, P2, and P3. Currently, in the page table, we have pages P1 and P3.



Demand Paging

So there are some steps that are followed in the working process of the demand paging in the operating system.

- **Program Execution:** When a program starts, the operating system creates a process for the program and allocates a portion of memory to the process.
- **Creating page tables:** The operating system creates page tables for processes, which track which program pages are currently in memory and which are on disk.
- **Page fault handling:** A page fault occurred when the program attempted to access a page that is not currently in memory. The operating system interrupts the program and checks the page tables to see if the required page is on disk.
- **Page Fetch:** If the required page is on disk, the operating system fetches the page from the disk and loads it into memory. The page table is then updated to reflect the page's new location in memory.
- **Resuming the program:** Once the required pages have been loaded into memory, the operating system resumes execution of the program where it left off. The program continues to run as if the page had always been in memory.
- **Page replacement:** If there is not enough free memory to hold all the pages a program needs, the operating system may need to replace one or more pages currently in memory with pages currently in memory. on the disk. The page replacement algorithm used by the operating system determines which pages are selected for replacement.
- **Page cleanup:** When a process terminates, the operating system frees the memory allocated to the process and cleans up the corresponding entries in the page tables.

Page Replacement Algorithms in Operating Systems

In an operating system that uses paging for memory management, a page replacement algorithm is needed to decide which page needs to be replaced when a new page comes in.

Page Fault: A page fault happens when a running program accesses a memory page that is mapped into the virtual address space but not loaded in physical memory. Since actual physical memory is much smaller than virtual memory, page faults happen. In case of a page fault, Operating System might have to replace one of the existing pages with the newly needed page. Different page replacement algorithms suggest different ways to decide which page to replace. The target for all algorithms is to reduce the number of page faults.

Page Replacement Algorithms:

1. First In First Out (FIFO): This is the simplest page replacement algorithm. In this algorithm, the operating system keeps track of all pages in the memory in a queue, the oldest page is in the front of the queue. When a page needs to be replaced page in the front of the queue is selected for removal.

Example 1: Consider page reference string 1, 3, 0, 3, 5, 6, 3 with 3 page frames. Find the number of page faults.

Page
reference

1, 3, 0, 3, 5, 6, 3

1	3	0	3	5	6	3
		0	0	0	0	3
	3	3	3	3	6	6
1	1	1	1	5	5	5
Miss	Miss	Miss	Hit	Miss	Miss	Miss

Total Page Fault = 6

Initially, all slots are empty, so when 1, 3, 0 came they are allocated to the empty slots —> **3 Page Faults**.

when 3 comes, it is already in memory so —> **0 Page Faults**. Then 5 comes, it is not available in memory so it replaces the oldest page slot i.e 1. —> **1 Page Fault**. 6 comes, it is also not available in memory so it replaces the oldest page slot i.e 3 —> **1 Page Fault**. Finally, when 3 come it is not available so it replaces 0 **1 page fault**.

Belady's anomaly proves that it is possible to have more page faults when increasing the number of page frames while using the First in First Out (FIFO) page replacement algorithm. For example, if we consider reference strings 3, 2, 1, 0, 3, 2, 4, 3, 2, 1, 0, 4, and 3 slots, we get 9 total page faults, but if we increase slots to 4, we get 10-page faults.

2. Optimal Page replacement: In this algorithm, pages are replaced which would not be used for the longest duration of time in the future.

Example-2: Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4 page frame. Find number of page fault.

Page reference 7,0,1,2,0,3,0,4,2,3,0,3,2,3

No. of Page frame - 4

7	0	1	2	0	3	0	4	2	3	0	3	2	3
			2	2	2	2	2	2	2	2	2	2	2
		1	1	1	1	1	4	4	4	4	4	4	4
	0	0	0	0	0	0	0	0	0	0	0	0	0
7	7	7	7	7	3	3	3	3	3	3	3	3	3
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit

Total Page Fault = 6

Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots —> **4 Page faults**

0 is already there so —> **0 Page fault**. when 3 came it will take the place of 7 because it is not used for the longest duration of time in the future.—>**1 Page fault**. 0 is already there so —> **0 Page fault**. 4 will takes place of 1 —> **1 Page Fault**.

Now for the further page reference string —> **0 Page fault** because they are already available in the memory.

Optimal page replacement is perfect, but not possible in practice as the operating system cannot know future requests. The use of Optimal Page replacement is to set up a benchmark so that other replacement algorithms can be analyzed against it.

3. Least Recently Used: In this algorithm, page will be replaced which is least recently used.

Example-3: Consider the page reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4 page frames. Find number of page faults.

Page reference 7,0,1,2,0,3,0,4,2,3,0,3,2,3

No. of Page frame - 4

7	0	1	2	0	3	0	4	2	3	0	3	2	3
			2	2	2	2	2	2	2	2	2	2	2
		1	1	1	1	1	4	4	4	4	4	4	4
	0	0	0	0	0	0	0	0	0	0	0	0	0
7	7	7	7	7	3	3	3	3	3	3	3	3	3
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit

Total Page Fault = 6

Here LRU has same number of page fault as optimal but it may differ according to question.

Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots
—> **4 Page faults**

0 is already there so —> **0 Page fault**. when 3 came it will take the place of 7 because it is least recently used —> **1 Page fault**

0 is already in memory so —> **0 Page fault**.

4 will take place of 1 —> **1 Page Fault**

Now for the further page reference string —> **0 Page fault** because they are already available in the memory.

4. Most Recently Used (MRU): In this algorithm, page will be replaced which has been used recently. Belady's anomaly can occur in this algorithm.

Page reference	7,0,1,2,0,3,0,4,2,3,0,3,2,3														No. of Page frame - 4	
7	0	1	2	0	3	0	4	2	3	0	3	2	3			
			2	2	2	2	2	2	3	0	3	2	3			
		1	1	1	1	1	1	1	1	1	1	1	1			
	0	0	0	0	3	0	4	4	4	4	4	4	4			
7	7	7	7	7	7	7	7	7	7	7	7	7	7			
Miss	Miss	Miss	Miss	Hit	Miss	Miss	Miss	Hit	Miss	Miss	Miss	Miss	Miss			
Total Page Fault = 12																

Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots
—> **4 Page faults**

0 is already there so—> **0 page fault**

when 3 comes it will take place of 0 because it is most recently used —> **1 Page fault**

when 0 comes it will take place of 3 —> **1 Page fault**

when 4 comes it will take place of 0 —> **1 Page fault**

2 is already in memory so —> **0 Page fault**

when 3 comes it will take place of 2 —> **1 Page fault**

when 0 comes it will take place of 3 —> **1 Page fault**

when 3 comes it will take place of 0 —> **1 Page fault**

when 2 comes it will take place of 3 —> **1 Page fault**

when 3 comes it will take place of 2 —> **1 Page fault**

Segmentation in Operating System

A process is divided into Segments. The chunks that a program is divided into which are not necessarily all of the exact sizes are called segments. Segmentation gives the user's view of the process which paging does not provide. Here the user's view is mapped to physical memory.

Types of Segmentation in Operating System

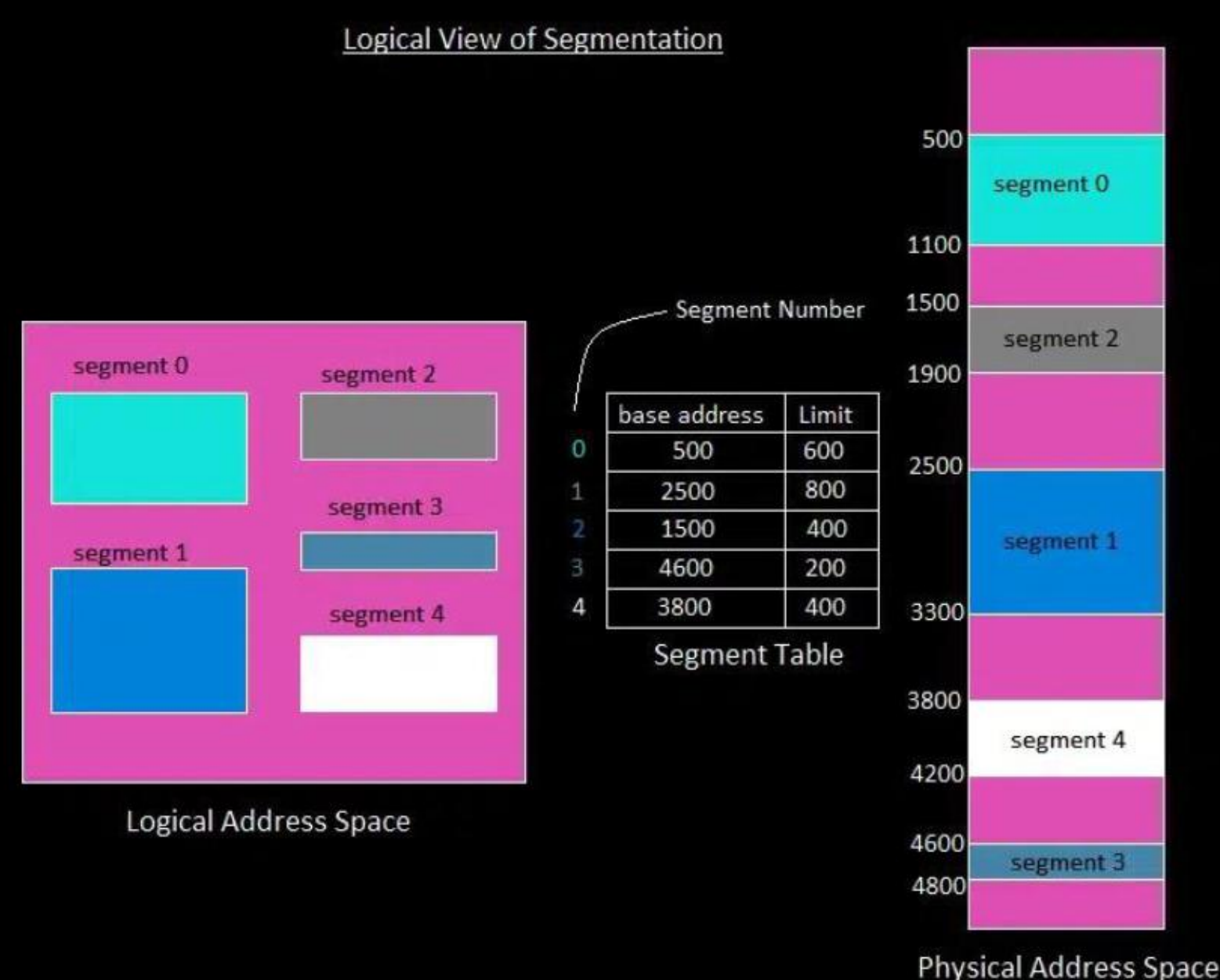
- **Virtual Memory Segmentation:** Each process is divided into a number of segments, but the segmentation is not done all at once. This segmentation may or may not take place at the run time of the program.
- **Simple Segmentation:** Each process is divided into a number of segments, all of which are loaded into memory at run time, though not necessarily contiguously.

There is no simple relationship between logical addresses and physical addresses in segmentation. A table stores the information about all such segments and is called Segment Table.

What is Segment Table?

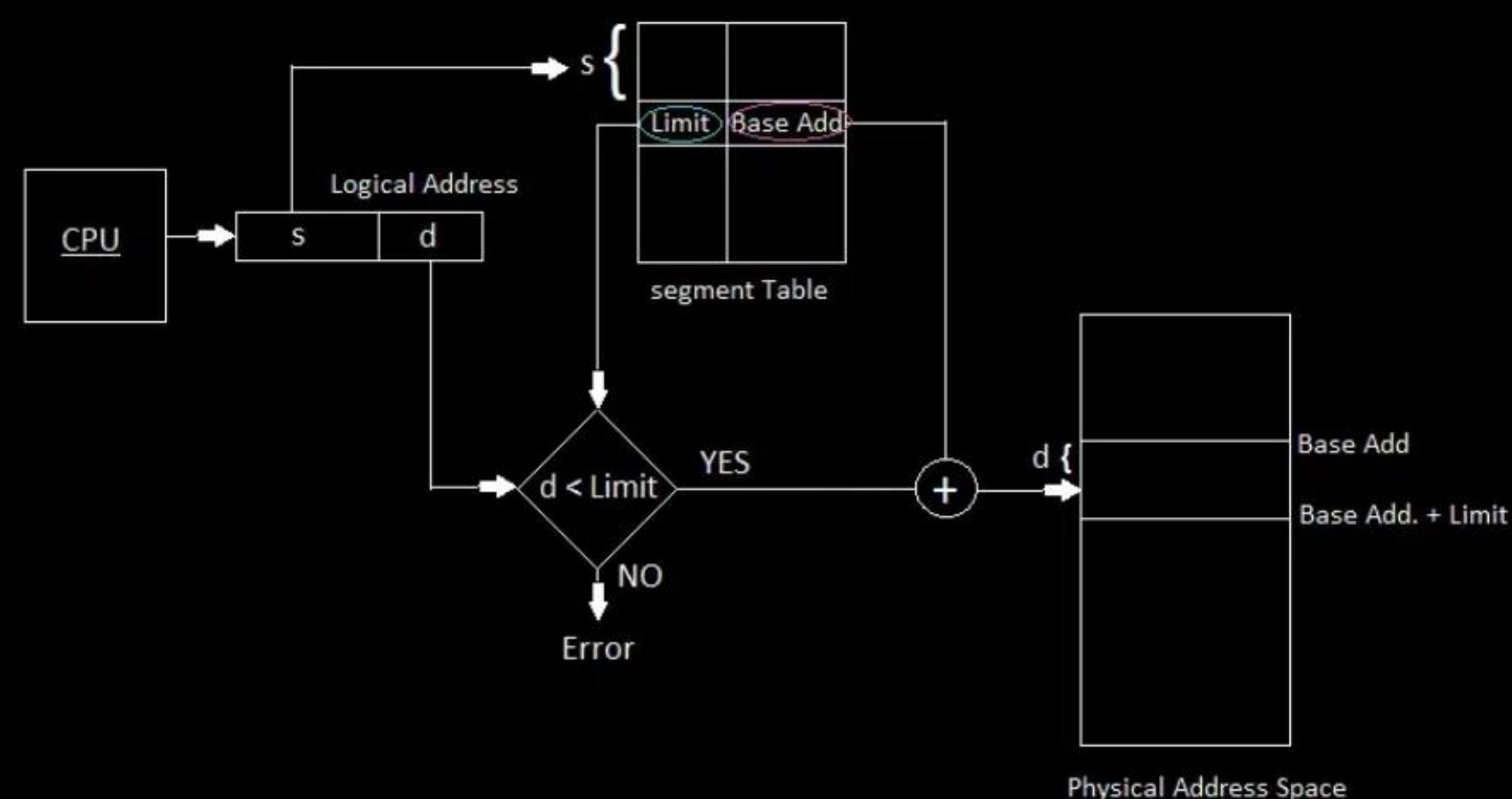
It maps a two-dimensional Logical address into a one-dimensional Physical address. It's each table entry has:

- **Base Address:** It contains the starting physical address where the segments reside in memory.
- **Segment Limit:** Also known as segment offset. It specifies the length of the segment.



Segmentation

Translation of Two-dimensional Logical Address to Dimensional Physical Address.



Translation

The address generated by the CPU is divided into:

- **Segment number (s):** Number of bits required to represent the segment.
- **Segment offset (d):** Number of bits required to represent the size of the segment.

Advantages of Segmentation in Operating System

- No Internal fragmentation.
- Segment Table consumes less space in comparison to Page table in paging.
- As a complete module is loaded all at once, segmentation improves CPU utilization.
- The user's perception of physical memory is quite similar to segmentation. Users can divide user programs into modules via segmentation. These modules are nothing more than separate processes' codes.
- The user specifies the segment size, whereas, in paging, the hardware determines the page size.
- Segmentation is a method that can be used to segregate data from security operations.
- **Flexibility:** Segmentation provides a higher degree of flexibility than paging. Segments can be of variable size, and processes can be designed to have multiple segments, allowing for more fine-grained memory allocation.
- **Sharing:** Segmentation allows for sharing of memory segments between processes. This can be useful for inter-process communication or for sharing code libraries.
- **Protection:** Segmentation provides a level of protection between segments, preventing one process from accessing or modifying another process's memory segment. This can help increase the security and stability of the system.

Disadvantages of Segmentation in Operating System

- As processes are loaded and removed from the memory, the free memory space is broken into little pieces, causing External fragmentation.
- Overhead is associated with keeping a segment table for each activity.
- Due to the need for two memory accesses, one for the segment table and the other for main memory, access time to retrieve the instruction increases.
- **Fragmentation:** As mentioned, segmentation can lead to external fragmentation as memory becomes divided into smaller segments. This can lead to wasted memory and decreased performance.
- **Overhead:** Using a segment table can increase overhead and reduce performance. Each segment table entry requires additional memory, and accessing the table to retrieve memory locations can increase the time needed for memory operations.
- **Complexity:** Segmentation can be more complex to implement and manage than paging. In particular, managing multiple segments per process can be challenging, and the potential for segmentation faults can increase as a result.

Non-Contiguous Allocation in Operating System

Non-contiguous allocation, also known as dynamic or linked allocation, is a memory allocation technique used in operating systems to allocate memory to processes that do not require a contiguous block of memory. In this technique, each process is allocated a series of non-contiguous blocks of memory that can be located anywhere in the physical memory.

Non-contiguous allocation involves the use of pointers to link the non-contiguous memory blocks allocated to a process. These pointers are used by the operating system to keep track of the memory blocks allocated to the process and to locate them during the execution of the process.

There are several advantages to non-contiguous allocation.

First, it reduces internal fragmentation since memory blocks can be allocated as needed, regardless of their physical location. Second, it allows processes to be allocated memory in a more flexible and efficient manner

since the operating system can allocate memory to a process wherever free memory is available.

However, non-contiguous allocation also has some disadvantages.

It can lead to external fragmentation, where the available memory is broken into small, non-contiguous blocks, making it difficult to allocate large blocks of memory to a process. Additionally, the use of pointers to link memory blocks can introduce additional overhead, leading to slower memory allocation and deallocation times.

In summary, non-contiguous allocation is a useful memory allocation technique in situations where processes do not require a contiguous block of memory. It is commonly used in operating systems, such as Unix and Linux, where processes often require variable amounts of memory that are not contiguous.

Some reference books on operating system concepts that cover non-contiguous allocation include:

1. "Operating System Concepts" by Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne.
2. "Modern Operating Systems" by Andrew S. Tanenbaum.
3. "Operating Systems: Three Easy Pieces" by Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau.
4. These books provide detailed coverage of operating system concepts, including memory management and allocation techniques.

Prerequisite – Variable Partitioning, Fixed Partitioning

Paging and Segmentation are the two ways that allow a process's physical address space to be non-contiguous. It has the **advantage** of reducing memory wastage but it increases the overheads due to address translation. It slows the execution of the memory because time is consumed in address translation.

In non-contiguous allocation, the Operating system needs to maintain the table which is called the **Page Table** for each process which contains the base address of each block that is acquired by the process in memory space. In non-contiguous memory allocation, different parts of a process are allocated to different places in Main Memory. Spanning is allowed which is not possible in other techniques like Dynamic or Static Contiguous memory allocation. That's why paging is needed to ensure effective memory allocation. Paging is done to remove External Fragmentation.

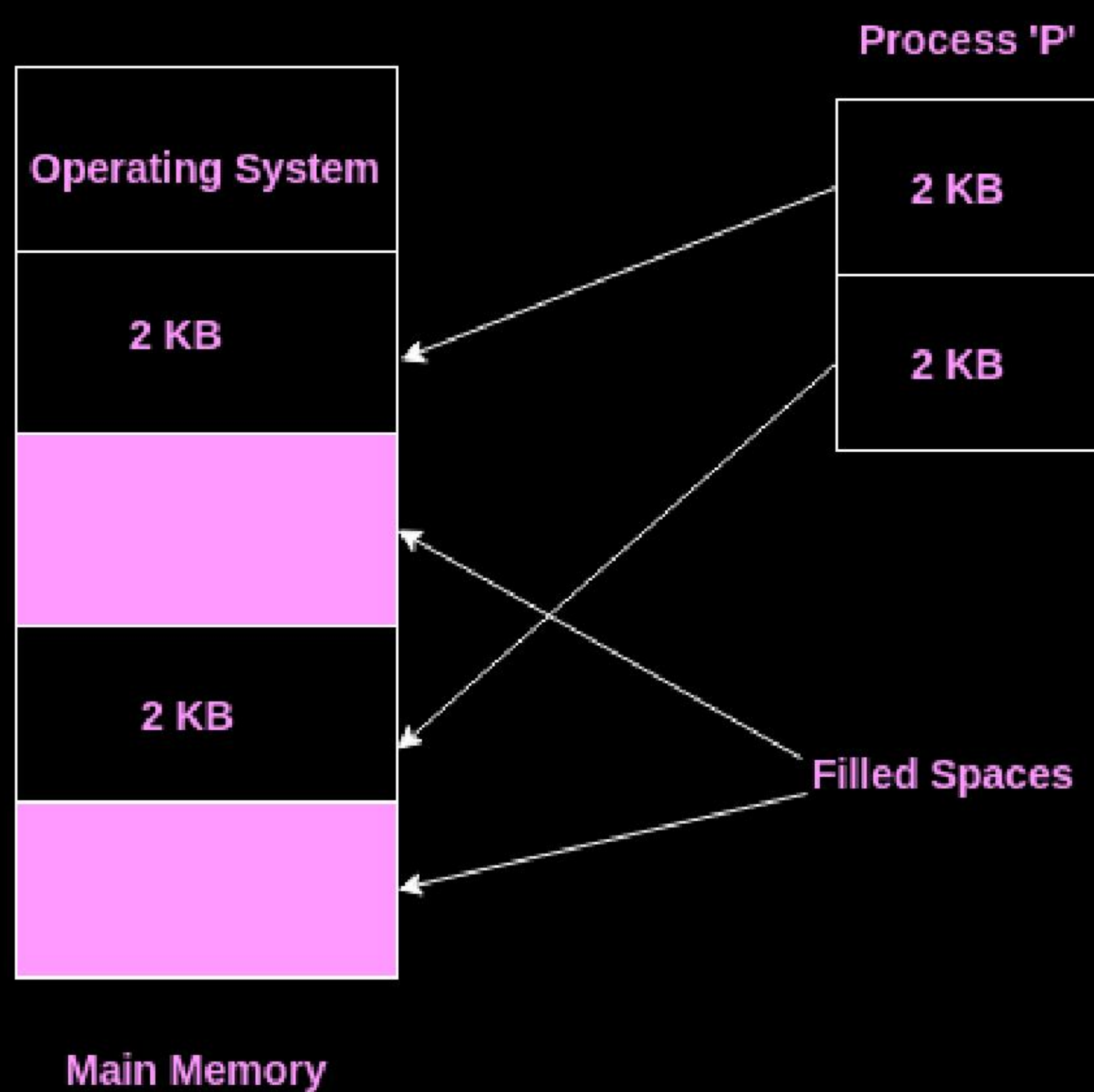
There are five types of Non-Contiguous Allocation of Memory in the Operating System:

1. Paging
2. Multilevel Paging
3. Inverted Paging
4. Segmentation
5. Segmented Paging

Working:

Here a process can be spanned across different spaces in the main memory in a non-consecutive manner. Suppose process P of size 4KB. Consider main memory has two empty slots each of size 2KB. Hence total free space is, $2 \times 2 = 4$ KB. In contiguous memory allocation, process P cannot be accommodated as spanning is not allowed.

In contiguous allocation, space in memory should be allocated to the whole process. If not, then that space remains unallocated. But in Non-Contiguous allocation, the process can be divided into different parts and hence filling the space in the main memory. In this example, process P can be divided into two parts of equal size – 2KB. Hence one part of process P can be allocated to the first 2KB space of main memory and the other part of the process can be allocated to the second 2KB space of main memory. The below diagram will explain in a better way:



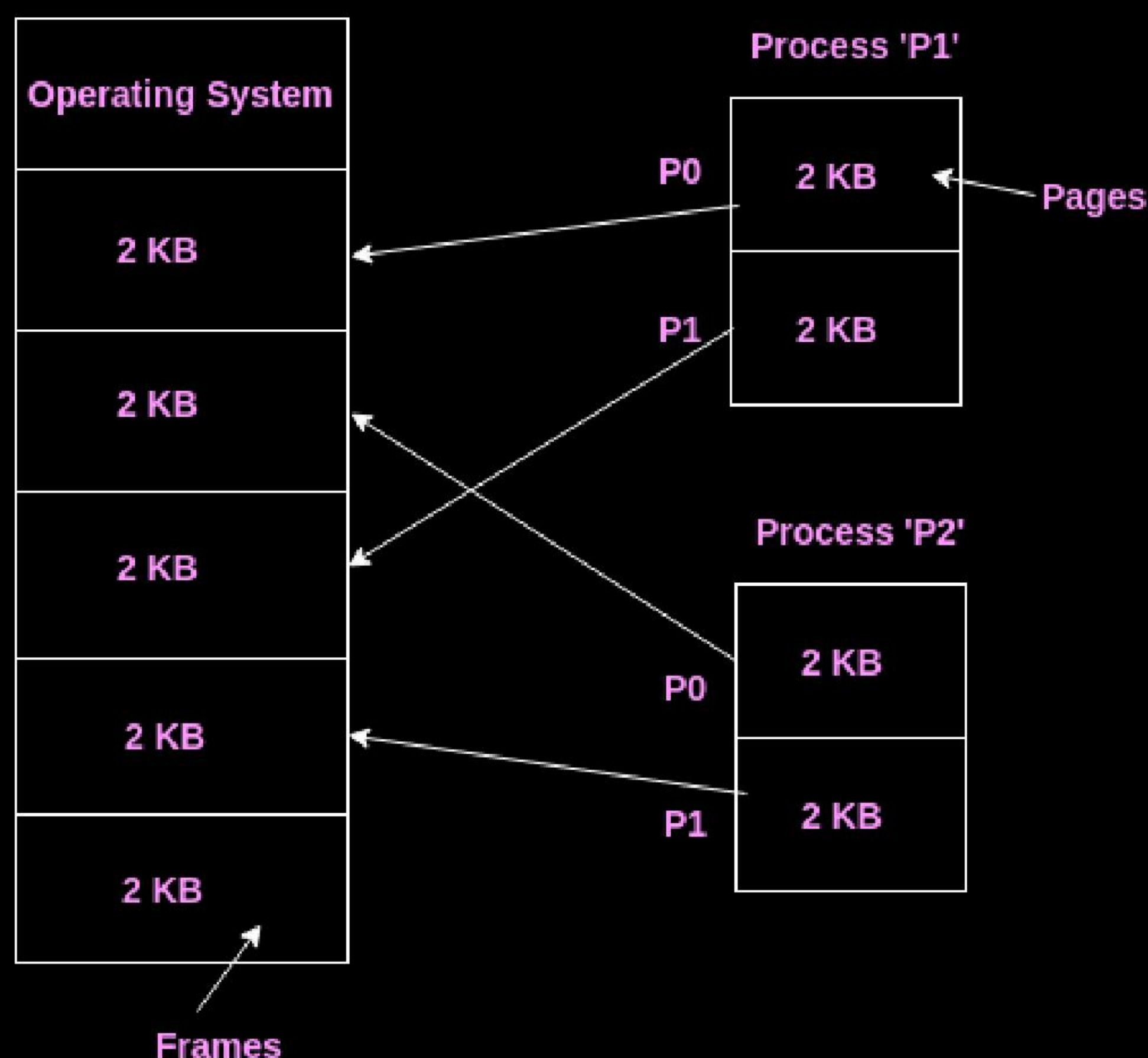
But, in what manner we divide a process to allocate them into main memory is very important to understand. The process is divided after analysing the number of empty spaces and their size in the main memory. Then only we do divide our process. It is a very time-consuming process. Their number as well as their sizes changing every time due to execution of already present processes in main memory.

In order to avoid this time-consuming process, we divide our process in secondary memory in advance before reaching the main memory for its execution. Every process is divided into various parts of equal size called Pages. We also divide our main memory into different parts of equal size called Frames. It is important to understand that:

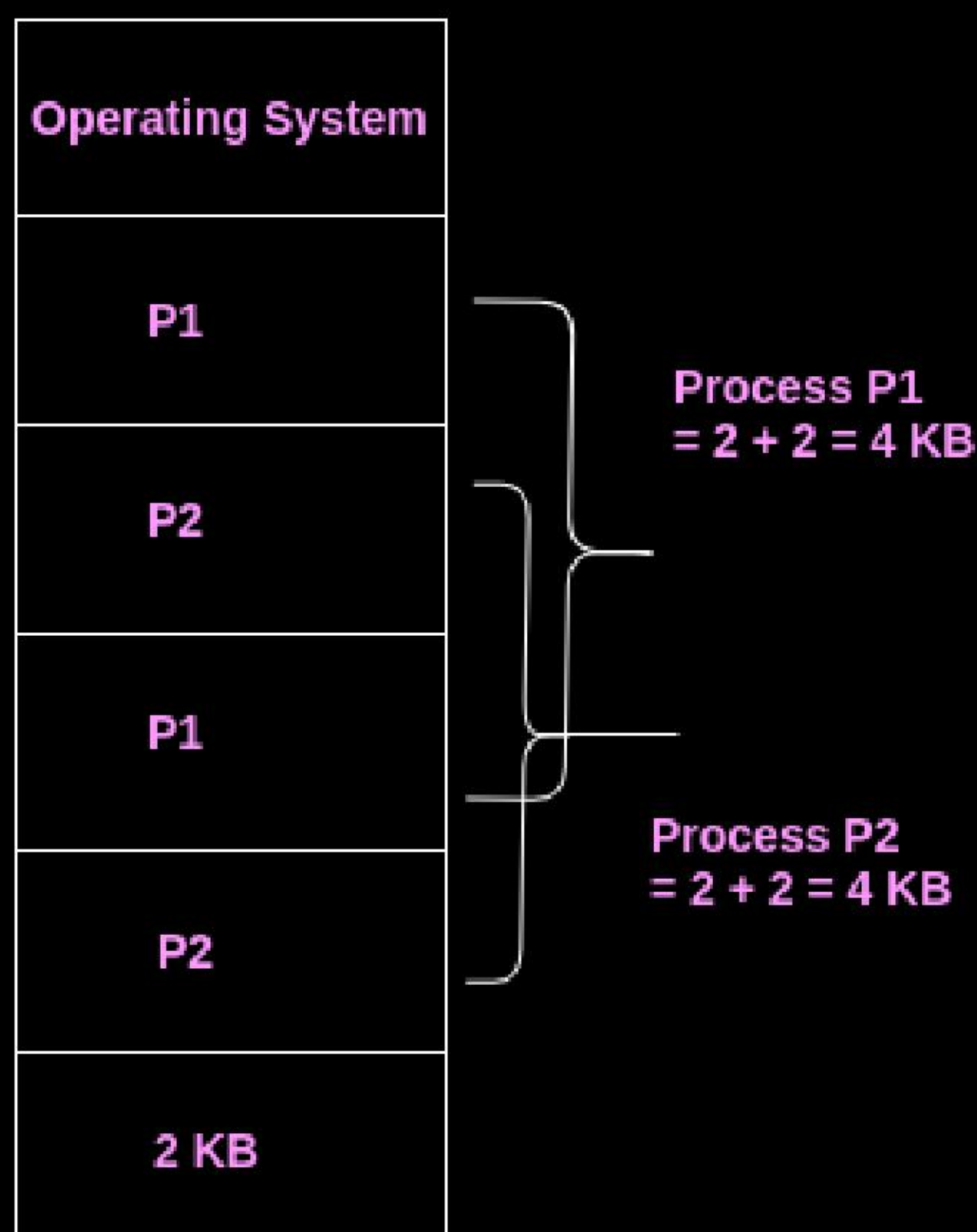
Size of page in process

= Size of frame in memory

Although their numbers can be different. Below diagram will make you understand it in a better way: consider empty main memory having a size of each frame is 2 KB, and two processes P1 and P2 are 2 KB each.



Resolvent main memory,



Concluding, we can say that Paging allows the memory address space of a process to be non-contiguous. Paging is more flexible as only pages of a process are moved. It allows more processes to reside in main memory than Contiguous memory allocation.

Paging in Operating System

Paging is a memory management scheme that eliminates the need for a contiguous allocation of physical memory. The process of retrieving processes in the form of pages from the secondary storage into the main memory is known as paging. The basic purpose of paging is to separate each procedure into pages. Additionally, frames will be used to split the main memory. This scheme permits the physical address space of a process to be non – contiguous.

In paging, the physical memory is divided into fixed-size blocks called page frames, which are the same size as the pages used by the process. The process's logical address space is also divided into fixed-size blocks called pages, which are the same size as the page frames. When a process requests memory, the operating system allocates one or more page frames to the process and maps the process's logical pages to the physical page frames.

The mapping between logical pages and physical page frames is maintained by the page table, which is used by the memory management unit to translate logical addresses into physical addresses. The page table maps each logical page number to a physical page frame number.

Terminologies Associated with Memory Control

- **Logical Address or Virtual Address:** This is a deal that is generated through the CPU and used by a technique to get the right of entry to reminiscence. It is known as a logical or digital deal because it isn't always a physical vicinity in memory but an opportunity for a connection with a place inside the device's logical address location.
- **Logical Address Space or Virtual Address Space:** This is the set of all logical addresses generated via a software program. It is normally represented in phrases or bytes and is split into regular-duration pages in a paging scheme.
- **Physical Address:** This is a cope that corresponds to a bodily place in reminiscence. It is the actual cope with this that is available on the memory unit and is used by the memory controller to get admission to the reminiscence.
- **Physical Address Space:** This is the set of all bodily addresses that correspond to the logical addresses inside the way's logical deal with place. It is usually represented in words or bytes and is cut up into fixed-size frames in a paging scheme.

In a paging scheme, the logical deal with the region is cut up into steady-duration pages, and every internet web page is mapped to a corresponding body within the physical deal with the vicinity. The going for walks tool keeps a web internet web page desk for every method, which maps the system's logical addresses to its corresponding bodily addresses. When a method accesses memory, the CPU generates a logical address, that is translated to a bodily address using the net page table. The reminiscence controller then uses the physical cope to get the right of entry to the reminiscence.

Important Features of Paging in PC Reminiscence Management

- **Logical to bodily address mapping:** In paging, the logical address area of a technique is divided into constant-sized pages, and each web page is mapped to a corresponding physical body within the main reminiscence. This permits the working gadget to manipulate the memory in an extra flexible way, as it is able to allocate and deallocate frames as needed.
- **Fixed web page and frame length:** Paging makes use of a set web page length, which is usually identical to the size of a frame within the most important memory. This facilitates simplifying the reminiscence control technique and improves device performance.

- **Page desk entries:** Each page within the logical address area of a method is represented through a page table entry (PTE), which contains facts approximately the corresponding bodily body in the predominant memory. This consists of the frame range, in addition to other manipulate bits which can be used by the running machine to manage the reminiscence.
- **A number of page desk entries:** The range of page desk entries in a manner's page desk is identical to the wide variety of pages inside the logical deal with the area of the technique.
- **Page table stored in important memory:** The web page desk for each system is typically saved in important reminiscence, to allow for green get right of entry to and change by the operating device. However, this could additionally introduce overhead, because the web page table must be updated on every occasion a system is swapped in or out of the main memory.

Example:

- If Logical Address = 31 bit, then Logical Address Space = 2^{31} words = 2 G words ($1\text{ G} = 2^{30}$)
- If Logical Address Space = 128 M words = $2^7 * 2^{20}$ words, then Logical Address = $\log_2 2^{27} = 27$ bits
- If Physical Address = 22 bit, then Physical Address Space = 2^{22} words = 4 M words ($1\text{ M} = 2^{20}$)
- If Physical Address Space = 16 M words = $2^4 * 2^{20}$ words, then Physical Address = $\log_2 2^{24} = 24$ bits

The mapping from virtual to physical address is done by the Memory Management Unit (MMU) which is a hardware device and this mapping is known as the paging technique.

- The Physical Address Space is conceptually divided into a number of fixed-size blocks, called **frames**.
- The Logical Address Space is also split into fixed-size blocks, called **pages**.
- Page Size = Frame Size

Let us consider an example:

- Physical Address = 12 bits, then Physical Address Space = 4 K words
- Logical Address = 13 bits, then Logical Address Space = 8 K words
- Page size = frame size = 1 K words (assumption)

Paging

The address generated by the CPU is divided into

- **Page number(p):** Number of bits required to represent the pages in Logical Address Space or Page number
- **Page offset(d):** Number of bits required to represent a particular word in a page or page size of Logical Address Space or word number of a page or page offset.

Physical Address is divided into

In a paging scheme, the physical space with the area is divided into fixed-length frames, each of which contains some bytes or words. When a program is running, its logical address space is split into constant-size pages, which might be mapped to corresponding frames within the physical address space.

To represent a physical address in this scheme, parts are commonly used:

Frame range: This is the variety of the frame within the physical space with the area that consists of the byte or phrase being addressed. The wide variety of bits required to represent the body range relies upon the scale of the

physical cope with the area and the size of each frame. For instance, if the physical cope with area carries 2^{20} frames and each frame is **4KB (2^{12} bytes)** in size, then the frame range could require $20-12 = 8$ bits.

Frame offset: This is the wide variety of the byte or word within the body this is being addressed. The number of bits required to represent the frame offset relies upon the size of every frame. For instance, if everybody is 4KB in size, then the frame offset could require 12 bits. So, a physical address in this scheme may be represented as follows:

Physical Address = (Frame Number $\ll$ Number of Bits in Frame Offset) + Frame Offset, where " $\ll$ " represents a bitwise left shift operation.

- The TLB is associative, high-speed memory.
- Each entry in TLB consists of two parts: a tag and a value.
- When this memory is used, then an item is compared with all tags simultaneously. If the item is found, then the corresponding value is returned.

Paging is a memory management technique used in operating systems to manage memory and allocate memory to processes. In paging, memory is divided into fixed-size blocks called pages, and processes are allocated memory in terms of these pages. Each page is of the same size, and the size is typically a power of 2, such as 4KB or 8 KB.

Important Points About Paging in Operating Systems

- **Reduces internal fragmentation:** Paging facilitates lessening internal fragmentation by using allocating memory in fixed-size blocks (pages), which might be usually a whole lot smaller than the size of the process's facts segments. This lets in for greater efficient use of memory in view that there are fewer unused bytes in each block.
- **Enables reminiscence to be allotted on call for:** Paging enables memory to be allocated on call for, this means that memory is most effectively allocated when it's far needed. This allows for extra efficient use of memory in view that only the pages that are absolutely used by the manner want to be allocated inside the physical memory.
- **Protection and sharing of memory:** Paging allows for the protection and sharing of reminiscence between methods, as each procedure has its own web page table that maps its logical deal with area to its physical address space. This permits techniques to proportion facts at the same time as preventing unauthorized get right of entry to every other's memory.
- **External fragmentation:** Paging can result in outside fragmentation, wherein memory turns fragmented into small, non-contiguous blocks. This can make it difficult to allocate

massive blocks of reminiscence to a method seeing that there may not be enough contiguous free memory to be had.

- **Overhead:** Paging involves overhead because of the renovation of the web page table and the translation of logical addresses to physical addresses. The working device must maintain the page table for each manner and perform a deal with translation whenever a procedure accesses memory, which can slow down the machine.

Virtual Memory in Operating System

Virtual Memory is a storage allocation scheme in which secondary memory can be addressed as though it were part of the main memory. The addresses a program may use to reference memory are distinguished from the addresses the memory system uses to identify physical storage sites and program-generated addresses are translated automatically to the corresponding machine addresses.

The size of virtual storage is limited by the addressing scheme of the computer system and the amount of secondary memory available not by the actual number of main storage locations.

It is a technique that is implemented using both hardware and software. It maps memory addresses used by a program, called virtual addresses, into physical addresses in computer memory.

All memory references within a process are logical addresses that are dynamically translated into physical addresses at run time. This means that a process can be swapped in and out of the main memory such that it occupies different places in the main memory at different times during the course of execution.

A process may be broken into a number of pieces and these pieces need not be continuously located in the main memory during execution. The combination of dynamic run-time address translation and the use of a page or segment table permits this.

If these characteristics are present then, it is not necessary that all the pages or segments are present in the main memory during execution. This means that the required pages need to be loaded into memory whenever required. Virtual memory is implemented using Demand Paging or Demand Segmentation.

Demand Paging

The process of loading the page into memory on demand (whenever a page fault occurs) is known as demand paging. The process includes the following steps are as follows:

Demand Paging

Demand Paging

If the CPU tries to refer to a page that is currently not available in the main memory, it generates an interrupt indicating a memory access fault.

The OS puts the interrupted process in a blocking state. For the execution to proceed the OS must bring the required page into the memory.

The OS will search for the required page in the logical address space.

The required page will be brought from logical address space to physical address space. The page replacement algorithms are used for the decision-making of replacing the page in physical address space.

The page table will be updated accordingly.

The signal will be sent to the CPU to continue the program execution and it will place the process back into the ready state.

Hence whenever a page fault occurs these steps are followed by the operating system and the required page is brought into memory.

Advantages of Virtual Memory

More processes may be maintained in the main memory: Because we are going to load only some of the pages of any particular process, there is room for more processes. This leads to more efficient utilization of the processor because it is more likely that at least one of the more numerous processes will be in the ready state at any particular time.

A process may be larger than all of the main memory: One of the most fundamental restrictions in programming is lifted. A process larger than the main memory can be executed because of demand paging. The OS itself loads pages of a process in the main memory as required.

It allows greater multiprogramming levels by using less of the available (primary) memory for each process.

It has twice the capacity for addresses as main memory.

It makes it possible to run more applications at once.

Users are spared from having to add memory modules when RAM space runs out, and applications are liberated from shared memory management.

When only a portion of a program is required for execution, speed has increased.

Memory isolation has increased security.

It makes it possible for several larger applications to run at once.

Memory allocation is comparatively cheap.

It doesn't require outside fragmentation.

It is efficient to manage logical partition workloads using the CPU.

Automatic data movement is possible.

Disadvantages of Virtual Memory

It can slow down the system performance, as data needs to be constantly transferred between the physical memory and the hard disk.

It can increase the risk of data loss or corruption, as data can be lost if the hard disk fails or if there is a power outage while data is being transferred to or from the hard disk.

It can increase the complexity of the memory management system, as the operating system needs to manage both physical and virtual memory.

Page Fault Service Time: The time taken to service the page fault is called page fault service time. The page fault service time includes the time taken to perform all the above six steps.

Let Main memory access time is: m

Page fault service time is: s

Page fault rate is : p

Then, Effective memory access time = $(p*s) + (1-p)*m$

Swapping

Swapping is a process out means removing all of its pages from memory, or marking them so that they will be removed by the normal page replacement process. Suspending a process ensures that it is not runnable while it is swapped out. At some later time, the system swaps back the process from the secondary storage to the main memory. When a process is busy swapping pages in and out then this situation is called thrashing.

Swapping

Swapping

Thrashing

At any given time, only a few pages of any process are in the main memory, and therefore more processes can be maintained in memory. Furthermore, time is saved because unused pages are not swapped in and out of memory. However, the OS must be clever about how it manages this scheme. In the steady state practically, all of the main memory will be occupied with process pages, so that the processor and OS have direct access to as many processes as possible. Thus when the OS brings one page in, it must throw another out. If it throws out a page just before it is used, then it will just have to get that page again almost immediately. Too much of this leads to a condition called Thrashing. The system spends most of its time swapping pages rather than executing instructions. So a good page replacement algorithm is required.

In the given diagram, the initial degree of multiprogramming up to some extent of point(λ), the CPU utilization is very high and the system resources are utilized 100%. But if we further increase the degree of multiprogramming the CPU utilization will drastically fall down and the system will spend more time only on the page replacement and the time taken to complete the execution of the process will increase. This situation in the system is called thrashing.

Causes of Thrashing

Thrashing

1. High Degree of Multiprogramming: If the number of processes keeps on increasing in the memory then the number of frames allocated to each process will be decreased. So, fewer frames will be available for each process. Due to this, a page fault will occur more frequently and more CPU time will be wasted in just swapping in and out of pages and the utilization will keep on decreasing.

For example:

Let free frames = 400

Case 1: Number of processes = 100

Then, each process will get 4 frames.

Case 2: Number of processes = 400

Each process will get 1 frame.

Case 2 is a condition of thrashing, as the number of processes is increased, frames per process are decreased. Hence CPU time will be consumed just by swapping pages.

2. Lacks of Frames: If a process has fewer frames then fewer pages of that process will be able to reside in memory and hence more frequent swapping in and out will be required. This may lead to thrashing. Hence a sufficient amount of frames must be allocated to each process in order to prevent thrashing.

Recovery of Thrashing

Do not allow the system to go into thrashing by instructing the long-term scheduler not to bring the processes into memory after the threshold.

If the system is already thrashing then instruct the mid-term scheduler to suspend some of the processes so that we can recover the system from thrashing.

Performance in Virtual Memory

Let p be the page fault rate($0 \leq p \leq 1$).

if $p = 0$ no page faults

if $p = 1$, every reference is a fault.

Effective access time (EAT) = $(1-p) * \text{Memory Access Time} + p * \text{Page fault time}$.

Page fault time = page fault overhead + swap out + swap in + restart overhead

The performance of a virtual memory management system depends on the total number of page faults, which depend on “paging policies” and “frame allocation”

Frame Allocation

A number of frames allocated to each process in either static or dynamic.

Static Allocation: The number of frame allocations to a process is fixed.

Dynamic Allocation: The number of frames allocated to a process changes.

Paging Policies

Fetch Policy: It decides when a page should be loaded into memory.

Replacement Policy: It decides which page in memory should be replaced.

Placement Policy: It decides where in memory should a page be loaded.